

Lecture 2 — Software Change & Concept Location

Lecture id: L02 **Source decks:** ChangeLec.pdf (= *Introduction to Software Change*, 14p), ChangeReqLec.pdf (= *Change Initiation*, 8p), ConceptLocation.pdf (*Concept Location*, 22p — slide footer counts 22, last page 21), JHotDraw.pdf (*Introduction to JHotDraw Framework*, 50p) **Labs:** ChangeReqLab.pdf (*CASE Study Lab — write a User Story*, 1p), CLLab1.pdf (*Concept Location Lab — IDE debugger / initial class set*, 1p) **Process phase(s):** Initiation · Change Request · Concept Location **Citation key:** slides cited as (ConceptLocation p.X), (JHotDraw p.X), (SoftwareChange p.X) for the *Introduction to Software Change* deck (file ChangeLec.pdf), (ChangeInitiation p.X) for the *Change Initiation* deck (file ChangeReqLec.pdf), (ChangeReqLab) / (CLLab) for labs; readings as [Raj13], [GHJV94], [Fowler99]. **Grounding note:** File-to-deck mapping verified against the actual PDFs — ChangeLec.pdf contains the *Introduction to Software Change* deck (14p, footers "X / 14") and ChangeReqLec.pdf contains the *Change Initiation* deck (8p, footers "X / 8"); filenames and titles are NOT swapped. Both decks are covered in full below. The slide author writes the framework name as "JHotDraw" but mistypes it "JHowDraw" once (ConceptLocation p.6). Citations [Raj13], [MC09], [Martin] do not appear verbatim on the slides; the slides cite Rajlich Ch.6 (2012 edition) and [GHJV94] (Gang-of-Four). Where this guide cites [Raj13] / [Fowler99] it is mapping slide content to the canonical course readings, not quoting a slide reference.

Overview

Lecture 2 is the first deep dive into the **canonical software change (SC) process** and introduces the **JHotDraw** case study that recurs across the whole course. It spans the first three phases of the eight-phase change process — **Initiation, Change Request, and Concept Location** — and previews all remaining phases so students see where these three sit in the whole. (SoftwareChange p.5)

Software change is defined as "*the process of adding new functionality to existing code*" and is named the **foundation of software evolution and servicing**. (SoftwareChange p.2) The lecture frames every change as flowing: **Stakeholders → Change Initiation → Change Request (in the Product Backlog) → Change → New Code plugged into Existing Code to reach Desired Code**. (ChangeInitiation p.8)

The full canonical process previewed here is: **Initiation → Concept Location → Impact Analysis → Prefactoring → Actualization → Postfactoring → Conclusion**, with **Verification** running vertically alongside Prefactoring through Conclusion. (SoftwareChange p.5) Three phases are grouped as *interactions with the world* (Initiation, Conclusion), *SC design* (Concept Location, Impact Analysis), and *SC implementation* (Prefactoring, Actualization, Postfactoring). (SoftwareChange p.5)

The bulk of the lecture's depth is **Concept Location** — the activity of finding the code snippet where a change must be made — taught both conceptually (the concept triangle, partial comprehension, methodologies) and procedurally (the **dependency search** algorithm walked through on Rajlich's UMLEditor example). The lecture closes by introducing **JHotDraw**, a Java drawing framework heavily built on **GoF design patterns**, which becomes the running codebase for labs and the portfolio assignment. (JHotDraw p.4, ChangeReqLab, CLLab)

Deck orientation — what each file contains and how the lecture flows

Four lecture decks plus two one-page lab handouts make up Lo2, all authored by **Jan Corfixen Sørensen**, **University of Southern Denmark**, under the course code **SB5-MAI** printed on every title slide. (SoftwareChange p.1, ChangeInitiation p.1, ConceptLocation p.1, JHotDraw p.1) The teaching sequence is:

1. **Introduction to Software Change** (file `ChangeLec.pdf`, 14 slides) — defines SC, classifies it along two independent axes (Lientz & Swanson motivation, impact on functionality), and previews **all eight elements of the phased model, one slide per element** (Initiation p.6, Design overview p.7, Concept Location p.8, Impact Analysis p.9, Prefactoring p.10, Actualization p.11, Postfactoring p.12, Verification p.13, Conclusion p.14). (SoftwareChange p.2–14)
2. **Change Initiation** (file `ChangeReqLec.pdf`, 8 slides) — zooms into phase 1: triggers, requirement forms, the user-story template with examples, the Product Backlog, and the end-to-end initiation diagram. (ChangeInitiation p.2–8)
3. **Concept Location** (file `ConceptLocation.pdf`, 21 physical slides; the slide footers count "/22") — zooms into phase 2: partial comprehension, the concept triangle, the watermark extraction example, the four methodologies, GREP, dependency search with the four marks, the nine-slide UML Editor trace, and the interactive Computer/Programmer tool. (ConceptLocation p.2–21)
4. **Introduction to JHotDraw Framework** (`JHotDraw.pdf`, 50 slides) — the case-study deck, organized by its own Outline slide into four parts: **JHotDraw** (p.3–7), **GoF Design Patterns** (p.8–14), **JHotDraw Design Patterns** (p.15–48), and **Other Design Patterns** (p.49–50, a section divider with no further content; p.50 is blank apart from the page number). (JHotDraw p.2)

A recurring visual anchor: every process slide in the change decks and the concept-location deck carries the same **chevron-stack sidebar** of the eight-element phased model, with the phase under discussion highlighted in orange/yellow and the VERIFICATION box drawn vertically beside Prefactoring through Conclusion — you can always read which phase a slide belongs to from that highlight. (SoftwareChange p.6–14, ChangeInitiation p.2, ConceptLocation p.2)

File-naming caution, verified against the actual files: `ChangeLec.pdf` holds the *Introduction to Software Change* deck (14 slides) and `ChangeReqLec.pdf` holds the *Change Initiation* deck (8 slides). This guide cites by deck name — (`SoftwareChange p.X`) and (`ChangeInitiation p.X`) — so every citation is unambiguous regardless of filename. (SoftwareChange p.1, ChangeInitiation p.1)

Learning Objectives

After this lecture a student should be able to:

1. **Define software change** and classify it by maintenance category (perfective, adaptive, corrective, preventive — Lientz & Swanson) and by impact on functionality (incremental, contraction, replacement, refactoring). (SoftwareChange p.2–4)
2. **Name and order the eight phases** of the canonical change process and explain which run as world-interactions vs. design vs. implementation, and that Verification spans the implementation phases. (SoftwareChange p.5–14)
3. **Describe Initiation**: what triggers a change, who the stakeholders are, and how requests are prioritized. (ChangeInitiation p.2–3)
4. **Write a change request as a User Story** in the form "*As a [user type], I want [goal] so that [reason]*", keep it to a 3"×5" card, and split it when too large. (ChangeInitiation p.4–6, ChangeReqLab)

5. **Explain the Product Backlog** as a prioritized "wish list" (Must / Nice-to-have / Won't have). (ChangeInitiation p.7)
 6. **Define concept location** and explain why change requests are formulated in domain-concept terms while code is not. (ConceptLocation p.2)
 7. **Apply the concept triangle** (Name, Intension, Extension) to reason about how a concept maps to code. (ConceptLocation p.4–5)
 8. **Extract concepts from a change request** (e.g., Export, Drawing, Text, Format from the watermark request). (ConceptLocation p.6)
 9. **Compare concept-location methodologies:** human knowledge, traceability tools, dynamic search (execution traces / IDE debugger), and static search (dependency search, pattern matching/GREP, information retrieval). (ConceptLocation p.7–8)
 10. **Execute the dependency search algorithm** over a Class Dependency Graph, using local vs. composite functionality and the Blank/Propagating/Unchanged/Next component marks, including backtracking. (ConceptLocation p.9–21)
 11. **Describe the JHotDraw architecture** (DrawApplication, StandardDrawingView, Drawing, Figure, Tool, Handle) and the **GoF design patterns** it embodies, and explain how those patterns make the framework changeable. (JHotDraw p.4–48)
 12. **Perform the lab tasks:** write a user story for a JHotDraw feature and produce an initial class set via the IDE debugger. (ChangeReqLab, CLLab)
-

Key Concepts

Software Change (SC) — definition and role

What it is. Software change is *"the process of adding new functionality to existing code"* and is named the **foundation of software evolution and servicing**. (SoftwareChange p.2) It is *not* a one-off coding task but a **named, repeatable engineering process** with a fixed set of phases. The visual metaphor on the slide is an "old system" arrow cutting over to a "new system" arrow that goes live — change is the engine that carries software from one running version to the next.

What it's used for / why it matters. SC exists because real-world software is never "finished": requirements, environments, and competitors keep moving, so the code must keep being modified without being rewritten from scratch. The reason it deserves a disciplined process is that modifying code you did not write — or wrote long ago — is **risk-laden**: you can break working behavior, introduce inconsistencies between dependent classes, or solve the wrong problem because you misread the request. The eight-phase model is the course's answer to that risk: it makes every change *traceable* (a request you can point to), *localized* (you change the minimum), and *verifiable* (you prove you didn't break anything). In the course's vocabulary, a single SC is one disciplined trip through the eight-phase process; **evolution** and **servicing** are simply the accumulation of many such trips over a product's life.

When & how it's applied. Every concrete activity in this course is an instance of SC. SC always operates on **existing code**, which distinguishes it from green-field development — this is the central premise of *software maintenance*. In the labs you start from the existing **JHotDraw** codebase, receive a request (e.g., add a watermark to exports), and walk it through the phases rather than building a drawing app from zero. The full change cycle (Initiation → ... → Conclusion + Verification) is *"the main topic of this course."* (SoftwareChange p.5)

Characteristics of SC — Lientz & Swanson maintenance categories

What it is. The classic **Lientz & Swanson** taxonomy classifies maintenance/change by *why* it is done — the motivation behind the request, not the mechanics of the edit. It sorts every change into exactly one of four buckets, with the slide giving an indicative effort distribution: (SoftwareChange p.3)

- **Perfective (~50%)** — improving/extending functionality for users; example: adding *credit card* support.
- **Adaptive (~25%)** — adapting to a changed *external* environment (new OS, hardware, library, regulation, or data format); example: the *Y2K* date fixes.
- **Corrective (~21%)** — fixing defects/*bugs* where the code already fails to meet its spec.
- **Preventive (~4%)** — improving *code structure* to forestall future problems before they bite (closely related to refactoring and reducing technical debt).

What it's used for / why it matters. The taxonomy is a planning and communication tool: it lets a team reason about *where their effort actually goes* and set expectations accordingly. The headline finding — only ~21% corrective — debunks the naïve assumption that "maintenance = bug fixing." Knowing that the bulk of work *adds or adapts* capability justifies why this course teaches a process built around *adding functionality to existing code* rather than a debugging-only workflow. It also helps you read a change request correctly: identifying the category tells you which downstream concerns dominate (a corrective change needs a failing test first; an adaptive change needs environment compatibility checks).

When & how it's applied. When a request lands in **Initiation**, you classify it. A user's "please add export-to-PDF" is perfective; "it crashes on a leap year" is corrective; "we must run on the new Java version" is adaptive; "let's untangle this God-class before the next feature" is preventive. The categories also map onto the *Initiation triggers*: a user reporting a bug → corrective; a user asking for an enhancement → perfective; a programmer proposing a cleanup → preventive (see Initiation phase).

Exam point: **Perfective dominates** — most maintenance effort adds or improves capability rather than fixing bugs, which motivates a change process tuned for *adding functionality to existing code* rather than just patching. (SoftwareChange p.3)

Impact on functionality — four kinds of change

What it is. This is the *second*, orthogonal classification: independently of *why* a change is made, it can be classified by *how* it affects the program's observable functionality. Where Lientz & Swanson answers "what motivated this?", this axis answers "what does the externally visible behavior look like before vs. after?": (SoftwareChange p.4)

- **Incremental** — adding *new* functionality that wasn't there (the set of behaviors grows).
- **Contraction** — removing obsolete functionality (the set of behaviors shrinks).
- **Replacement** — swapping existing functionality for a different implementation/behavior of the same feature.
- **Refactoring** — changing software *structure* **without changing behavior** at all (the externally observable behavior is identical before and after — the definition repeated throughout the course; see Prefactoring/Postfactoring). (SoftwareChange p.4)

What it's used for / why it matters. This classification matters because **the four kinds carry very different verification obligations**. An incremental change needs *new* tests for the new behavior; a refactoring must pass the *existing* tests unchanged (that is literally how you prove behavior didn't change); a replacement needs both the old tests retired and new ones written; a contraction needs you to confirm nothing still depends on the removed feature. Recognizing the kind up front tells you what "done" looks like.

When & how it's applied. The watermark request is *incremental* (new behavior on export). Deleting a deprecated file format is *contraction*. Rewriting the export engine to produce the same files faster is *replacement*. Splitting a bloated `DrawApplication` into smaller classes before adding a feature is *refactoring* — and crucially the two refactoring phases of the process (**Prefactoring** and **Postfactoring**) are exactly this kind applied at two moments.

Note the conceptual link across the two classifications: *Preventive maintenance* (the *why*) is typically realized as *Refactoring* (the *how*) — they are the same activity seen from the motivation axis and the behavior axis respectively.

The phased model of software change (the canonical process)

What it is. The course's spine is the eight-element phased model — a fixed, ordered recipe that takes a change request and turns it into a committed new baseline. Each phase has one job and hands a well-defined artifact to the next, so the process is essentially a pipeline. Reading top-to-bottom: (SoftwareChange p.5–14)

1. **Initiation** — a change request arrives; requirements captured and prioritized. (world interaction)
2. **Concept Location** — extract concepts from the request and find them in the code. (design)
3. **Impact Analysis** — determine strategy and the *impact set* of classes affected. (design)
4. **Prefactoring** — opportunistic refactoring *before* the change to localize/minimize its impact. (implementation)
5. **Actualization** — create the new code and plug it into the old code; propagate changes to neighbors. (implementation)
6. **Postfactoring** — refactor *after* the change to remove anti-patterns introduced by it. (implementation)
7. **Conclusion** — commit to version control, build the new baseline, prepare for next change. (world interaction)
8. **Verification** — runs *vertically* alongside Prefactoring → Conclusion, guaranteeing correctness throughout (testing + walkthroughs). (SoftwareChange p.5, p.13)

What it's used for / why it matters. The model is the discipline that keeps maintenance from degenerating into ad-hoc "edit until it seems to work." Its value is in the *ordering and the hand-offs*: you find *where* to change (Concept Location) before deciding *how far* the change reaches (Impact Analysis) before you *make room* for it (Prefactoring), and you never integrate without *proving correctness* (Verification). Because the phases produce concrete artifacts — a user story, an initial class set, an impact set, refactored code, a commit — the process is also what the **portfolio assignment** is built around. The "world interaction / design / implementation" grouping tells you *what kind of activity* each phase is: talking to people, reasoning about code on paper, or actually editing code.

When & how it's applied. Across the course, a single feature (e.g., the JHotDraw watermark) is carried through all eight phases; this lecture executes phases 1–2 fully and later lectures pick up 3–8. The eighth element, **Verification**, is drawn *vertically* on the slide precisely because it is not a step you do once at the end — it overlaps Prefactoring through Conclusion so that every implementation move is checked as it happens.

Grouping: *Interactions with the world* (Initiation, Conclusion) · *SC design* (Concept Location, Impact Analysis) · *SC implementation* (Prefactoring, Actualization, Postfactoring). (SoftwareChange p.5) **L02 covers phases 1–2 in depth (Initiation + Concept Location) and previews 3–8.**

Initiation phase

What it is. Initiation is the **first phase** and the only one that is purely an *interaction with the world*: it is where a need outside the code becomes a recorded, prioritized work item. **SC starts with a change request.** (SoftwareChange p.6, ChangeInitiation p.2) The phase has two jobs — *capture* the requirement and *prioritize* it among everything else waiting. Triggers come from different stakeholders: (ChangeInitiation p.3)

- a **user reports a software bug** (→ corrective),
- a **user asks for an enhancement** (→ perfective),
- a **programmer proposes an improvement** (→ preventive/perfective),
- a **manager wants to meet a competitor's functionality** (→ perfective/business-driven).

What it's used for / why it matters. Initiation is the funnel that prevents two failure modes: working on the wrong thing, and working on things in the wrong order. By forcing every change to begin as an explicit, written request, it creates the *traceability* the whole process depends on — every later artifact (located classes, impact set, commit) can be traced back to this request. The **prioritization** step matters because teams always have more requests than capacity; it decides what gets pulled into the next change cycle. Identifying the trigger also seeds the **Lientz & Swanson** classification, which shapes how the rest of the process treats the change.

When & how it's applied. In the lab, Initiation is where you pick a JHotDraw feature, write its request, and drop a card into the backlog. The requirement may be a software bug, an enhancement, or an improvement. (ChangeInitiation p.2) The mnemonic shown on the slide is the user-story sticky note: "*As a [who], I want [what], because [why].*" (ChangeInitiation p.3) — the format that immediately follows in the next concept.

Change Request and requirements form

What it is. The **change request** is the *artifact* produced by Initiation — the recorded statement of what is wanted. It is the unit of work the rest of the process consumes. A change request can take several **forms**, chosen to match the nature of the request: (ChangeInitiation p.4)

- a **sentence or paragraph** — fine for small, unambiguous asks,
- a **bug report** — for corrective changes, capturing symptom/steps/expected vs. actual,
- a **user story** — preferred for new functionality, because it pins down *who* benefits and *why*.

What it's used for / why it matters. The form is not cosmetic: it controls how much ambiguity survives into the design phases. A **user story** should *limit complexity and the potential for misunderstanding*; it must **fit on a 3"×5" card**, and *if new functionality cannot fit, it has to be divided into several user stories*. (ChangeInitiation p.4) The card-size constraint is a deliberate **complexity governor** — a physical limit that keeps each change small enough to flow cleanly through one pass of the change process, be located, impact-analyzed, and verified without the scope ballooning. A request too big to fit is a signal that it is really several changes wearing one label.

When & how it's applied. For the watermark feature you would *not* write a bug report (nothing is broken); you write a user story because it is new, user-facing functionality. If "add a watermark to every export format" turns out too large for one card, you split it — for instance one story per export format — and each becomes its own trip through the process. Choosing the form correctly is the first concrete decision a maintainer makes after a request arrives.

User Story — definition and template

What it is. A **user story** is a *"short and simple description of a capability written from the perspective of the person who desires the new capability."* (ChangeInitiation p.5) Its defining feature is the *point of view*: it is told from the user's side, naming the actor, their goal, and their motivation — never the implementation. Canonical templates:

- *"As a [user type], I want [some goal] so that [some reason]."* (ChangeInitiation p.5)
- Alternative: *"As a [user type], I want [some goal] because [why]."* (ChangeInitiation p.5)

What it's used for / why it matters. The three slots do real work. The **[user type]** identifies *who* to validate the change with and whose perspective defines "correct." The **[goal]** is the capability to build. The **[so that / because]** captures the *value* — the reason that, if a shortcut tempts you, tells you what must not be lost, and that helps prioritize and split the story. By keeping the description in the *problem space* (what the user needs) rather than the *solution space* (how to code it), the story leaves the design open and reduces the chance of building the wrong thing. It is also the source of the **domain concepts** that Concept Location later extracts.

When & how it's applied. You write one whenever the change adds new, user-facing functionality. Worked examples from the deck: (ChangeInitiation p.6) - *"As a [valid user], I want [to access the system] so that [I can review my information]."* - *"As an [administrator], I want to [restrict access to the system to valid users] so that [I can ensure we protect user information]."*

For JHotDraw you pick an existing feature and back-write its story (e.g., the watermark story in the Worked Example). The user story is the artifact the student must produce for the portfolio (see Worked Example and the lab). (ChangeReqLab)

The two user-story examples dissected (ChangeInitiation p.6)

What they are. The deck gives exactly two worked stories, and each slot repays close reading: (ChangeInitiation p.6)

1. *"As a [valid user], I want [to access the system] so that [I can review my information]."* — Actor: *valid user* (note the qualifier — not just any user, but one already entitled to access). Goal: system access. Reason: reviewing *their own* information, which scopes the access narrowly.
2. *"As an [administrator], I want to [restrict access to the system to valid users] so that [I can ensure we protect user information]."* — Actor: *administrator* (a different user type with a different stake). Goal: restricting access. Reason: protecting user information — a security/duty motivation rather than a personal-benefit one.

Why the pair is instructive. The two stories are *complementary views of the same access-control feature*: the valid user wants the door to open; the administrator wants the door to stay closed to everyone else. Same system capability, two user types, two goals, two reasons — demonstrating why the **[user type]** slot exists at all: the perspective changes what "the feature" even means and who validates it. The pair also models the reason-slot's range: "so that I can review my information" is a value-to-self reason, "so that I can ensure we protect user information" is a value-to-others/duty reason — both legitimate fillers of the *so that* slot. (ChangeInitiation p.5–6)

When & how it's applied. When back-writing a story for an existing JHotDraw feature in the lab, ask whose perspective makes the feature meaningful — and check whether a *second* stakeholder's story exists for

the same feature, as it does here. If your single card is really two stakeholders' stories fused together, that is the story-splitting rule firing on the actor axis rather than the size axis. (ChangeInitiation p.4, ChangeReqLab)

Product Backlog

What it is. The **Product Backlog** is a prioritized "*wish list*" of requirements — a single, ordered, living list of everything the product might do next. Stakeholders **add / delete / modify** items as understanding evolves. (ChangeInitiation p.7) Items are prioritized in tiers — **Must have / Nice to have / Won't have** — a MoSCoW-style stacking that says not just *whether* but *how urgently* each item is wanted. (ChangeInitiation p.7)

What it's used for / why it matters. The backlog is the team's **shared queue and single source of truth for "what's next."** It solves the coordination problem of a stream of requests arriving from many stakeholders at different times: instead of competing channels and forgotten asks, everything lands in one ranked list that the team pulls from top-down. It is deliberately *mutable* — it exists precisely because requirements are not known up front: *additional knowledge is acquired by the users* and *additional clarification is needed by the developers* over time, so items are continuously reprioritized, refined, and dropped. (ChangeInitiation p.7) The tiering also makes the cut line explicit: "Won't have" is an honest decision, not a backlog item left to rot.

When & how it's applied. Each user story you write becomes a **card** placed in the backlog (in the lab, a card in GitHub Projects' TODO column). In the end-to-end picture the backlog is the **container that holds the Change Request** between Initiation and the actual Change — the request waits there, prioritized, until the team starts the change cycle on it. (ChangeInitiation p.8) The watermark story, for example, likely sits in the "Nice to have" tier until pulled.

Concept Location — definition and motivation

What it is. **Concept location** "*finds the code snippet where a change is to be made.*" (ConceptLocation p.2) It is the first *design* phase: a search activity that takes a domain concept from the request and pins it to a specific place — a class or method — in the existing code. The output is a *starting point*, not the whole change.

What it's used for / why it matters. It exists to bridge a **vocabulary gap: change requests are most often formulated in terms of domain concepts** (the words users and managers use), but those concepts are scattered across the code, renamed by programmers, split across classes, or buried inside methods. (ConceptLocation p.2) Without a deliberate location step you cannot even begin a change on an unfamiliar codebase — you would not know which of thousands of classes to open. Concept location is therefore the activity that turns "what the user said" into "the line of code I must touch," and it is the prerequisite for every later phase: you cannot analyze impact, prefactor, or actualize until you know *where* the change lands.

When & how it's applied. Example request from the slide: "*Correct error that arises when trying to paste a text*" — the programmer must locate where *paste* and *text* live in the code before fixing anything. (ConceptLocation p.2) In the process, Concept Location follows Initiation and feeds Impact Analysis: "*Concepts are extracted from the change request*" and the "*extracted concepts are located in the code and used as a starting point of SC.*" (SoftwareChange p.8) The classes found here form the **initial impact set** consumed by Impact Analysis. (SoftwareChange p.9) In the lab you do this on JHotDraw with the IDE debugger, producing a Domain-Class/Responsibility table.

Partial code comprehension (as-needed strategy)

What it is. Partial code comprehension is the *cognitive stance* concept location takes: the acceptance that **large programs cannot be completely comprehended** by any one person, so the programmer deliberately aims for only the **minimum essential understanding** needed for the task at hand. (ConceptLocation p.3) It uses an **as-needed strategy** — building understanding of *how certain specific concepts are reflected in the code* rather than mastering the whole system. (ConceptLocation p.3)

What it's used for / why it matters. This is the justification for why concept location is even a sensible idea. If complete understanding were feasible you would just read the whole system and know where everything is; in reality, codebases are too large and memory too limited, so trying to understand everything is both impossible and wasteful. The as-needed strategy keeps the maintainer's effort proportional to the change: a small change should require understanding only a small slice of code. It also frames *partial* understanding as legitimate engineering, not a shortcut — you are *expected* to change code you do not fully understand, guided by the located concept.

When & how it's applied. The slide's analogy is **visiting a large city** — you learn the route to your destination, not every street. (ConceptLocation p.3) Applied to JHotDraw (hundreds of classes), you do not read the whole framework to add a watermark; you trace from the export action just deep enough to find the class that writes the file, understand that one path, and stop. The dependency-search algorithm and the IDE-debugger lab are both *mechanisms* for executing this as-needed navigation.

The Concept Triangle (Name / Intension / Extension)

What it is. The concept triangle is a model of *what a concept is made of*, borrowed from semiotics. A **concept** has three corners and the relationships between them: (ConceptLocation p.4)

- **Name** — the label/identifier used to refer to the concept (a word or symbol).
- **Intension** — the *definition* of the concept: its defining properties and meaning (what makes something an instance).
- **Extension** — the *set of all instances* the concept denotes (the actual things it picks out in the world or in code).

The edges name the *operations* that connect the corners: - **Name** → **Intension** = **naming / definition** (a name is given meaning by a definition). - **Name** → **Extension** = **annotation / traceability** (a name is attached to, and traces to, instances). - **Intension** → **Extension** = **recognition / location** (from a definition you *recognize* and *locate* the instances that satisfy it).

What it's used for / why it matters. The triangle is the *theoretical justification* for why concept location is hard and how to do it anyway. The trap in maintenance is to search by **Name** alone (grep the request's words) — but the name in the request often differs from the name in the code (synonyms, translations, abbreviations, internal jargon). The triangle says: don't rely on the name edge; rely on **Intension** → **Extension** (**recognition/location**). You hold the *meaning* of the concept in your head and recognize the code that implements that meaning, even when it is named differently. This is exactly the move dependency search makes when it asks "is *this concept* implemented here?" rather than "does the word appear here?"

When & how it's applied. Worked illustration from the deck: the **Name** "Dog / Pes / Hund" (same concept, three natural-language names — the point being that the name is not stable) has **Intension** "Hairy animal with teeth..." and **Extension** = {Fido, Lassie, Buck (from *Call of the Wild*), ..., a generic cartoon dog}. (ConceptLocation p.5) Applied to the watermark request: the *intension* of "Export" ("turning a drawing into

an external file format") lets you recognize the code that exports even if the class is called `StorageFormat` or `ImageWriter` rather than `Export` — you reason from meaning to the actual code instances.

Extracting concepts from a change request

What it is. Concept extraction is the *first sub-step* of concept location: reading the change request and identifying the **domain concepts** — typically the meaningful domain nouns — that the change is about. It converts a prose request into a short list of named targets to find in the code.

What it's used for / why it matters. It scopes the search. Each extracted concept becomes a separate location target, so the list tells you *how many* things you must find and *what they are*. Doing this explicitly stops you from missing part of a request (e.g., remembering "watermark text" but forgetting it must work for "all formats") and gives you the vocabulary you then reason about through the concept triangle. The extracted concepts are also what you record as you search and ultimately what you hand to Impact Analysis.

When & how it's applied. You underline the domain concepts in the request. Worked example change request: (ConceptLocation p.6)

"Modify the export feature of JHotDraw to automatically include a simple watermark text in the drawings being exported. ... The watermark should be included uniformly for all possible export formats."

Concepts extracted: Export, Drawing, Text, Format. (ConceptLocation p.6) These four become the search terms/targets that concept location must find in the code — you would then locate the class that performs *Export*, confirm where *Drawings* and their *Text* live, and check that whatever you change applies uniformly across every *Format*.

Concept location methodologies

What it is. These are the four families of *techniques* for actually finding a concept in code — the toolbox you choose from once you know what you are looking for. They differ along two axes: *who/what does the locating* (a person, a tool, the running program, or static analysis) and *whether the program runs*. (ConceptLocation p.7)

1. **Human knowledge** — a programmer who already knows the code simply points to the location. *Fastest when available, but depends entirely on having an expert and on their memory being current.*
2. **Traceability tools** — pre-maintained links between requirements/concepts and the code that implements them. *Instant lookup when the links exist, but only as good as the discipline that kept them up to date.*
3. **Dynamic search (execution traces)** — run the program, exercise the concept, and observe which code executes (the basis of the IDE-debugger lab). *Pinpoints code that actually runs for the feature, but only finds paths you manage to trigger.* (CLLab)
4. **Static search** — locate without running the code: **dependency search** (follow the Class Dependency Graph), **pattern matching** (e.g., GREP over the text), and **information retrieval** techniques (rank files by textual relevance). *Works without a runnable build and covers all code, but can be fooled by naming and dead code.*

What it's used for / why it matters. Naming the four families lets you pick the *cheapest method that will work* for your situation and combine them. They are complementary: human knowledge or traceability if you are lucky enough to have them; dynamic search when you can run the feature and want to see real execution;

static search when you cannot run it or want the structural map. Knowing the trade-offs is itself an exam point — e.g., dynamic search misses untriggered branches, while GREP misses renamed concepts.

When & how it's applied. For the JHotDraw watermark you would likely have no expert and no traceability links, so you combine **dynamic search** (run JHotDraw, trigger an export, watch which classes execute — the CLLab approach) with **static dependency search** (follow suppliers from the export action) and perhaps seed candidates with **GREP** for `export / write / format`.

GREP search technique (static / pattern matching)

What it is. **GREP** = "*global regular expression print*." (ConceptLocation p.8) It is the canonical **pattern-matching** static-search tool: given a regular expression, it **prints the lines that contain a match**. (ConceptLocation p.8) It treats source code as plain text and finds wherever a pattern literally appears.

What it's used for / why it matters. GREP is the quick, zero-setup way to *seed* a search — it answers "where does this string occur?" across the whole codebase in seconds, with no build, no graph, and no running program. Its weakness is the flip side of its simplicity: because it matches *text*, it is **blind to dependencies and to renaming**. It cannot follow a method call into the class that really does the work, and it cannot find a concept that the code names differently from your query (the concept-triangle name-edge trap). That is precisely why the course pairs it with **dependency search**, which follows structure rather than text.

When & how it's applied. Usage is **iterative**: the programmer formulates a search query, investigates the results, and refines the query. (ConceptLocation p.8) For the watermark task you might `grep` for `export`, then `write`, then `format`, narrowing the regex as you learn the code's naming, to produce a shortlist of candidate files — then switch to dependency search to trace the real export path. GREP gets you in the door; it rarely finishes the job alone.

Dependency search technique

What it is. Dependency search is a **static** location method that navigates the code's structure rather than its text. It uses **Class Dependency Graphs (CDG)** — graphs whose nodes are classes and whose edges are "depends on / uses" relationships — **extracted from the existing code**. (ConceptLocation p.9) You walk this graph from a starting class along dependency edges, deciding at each class whether to go deeper. Two notions of a module's functionality drive every decision: (ConceptLocation p.9)

- **Local functionality** — the concepts *actually implemented in the module itself*, **not delegated** to others (what *this class* really does on its own).
- **Composite functionality** — the *complete* functionality of a module **combined with all its supporting (supplier) modules** (what the class delivers *through everything it calls*).

What it's used for / why it matters. Dependency search fixes GREP's blind spot: by following call/use edges it can locate a concept even when the responsible class is named nothing like the request, because it reasons about *what classes do* (intension → extension) instead of *what they are called*. The **local vs. composite** distinction is the engine of the method and the single most important idea here: it tells you whether you have arrived or must keep going. If a concept is present in a class's **composite** functionality, the feature *flows through* this class — but if it is not in the **local** functionality, this class is only a *conduit*, delegating the real work to a supplier, so you must descend. Only when the concept is in the **local** functionality have you found the place to change.

When & how it's applied. These functionalities are *determined by reading code and documentation*. (ConceptLocation p.9) Concretely: starting at the export Tool, you read it and ask "does this class itself

stamp/format the export (local), or does it just call something that does (composite)?" If the export action merely delegates to a format-writer, *Export* is in its composite but not its local responsibility — so you follow the edge to the writer. Local vs. composite is the **decision pivot** in the search algorithm below: composite-but-not-local ⇒ descend into suppliers; local ⇒ stop.

The dependency-search algorithm

What it is. This is the *procedure* that operationalizes dependency search — a small decision loop, drawn as an activity diagram and also as an interactive Computer/Programmer swim-lane tool, that you repeat class-by-class until the concept is found: (ConceptLocation p.10, p.21)

1. **Find the set of starting modules** (computer-supported). Features often start from **controller** classes. (CLLab)
2. **Select one module** to inspect.
3. **Is the concept implemented in the (local) module?** — [Yes] → **stop the search** (concept located here).
4. If [No], ask: **Is the concept implemented in the composite responsibility?** - [Yes] → **find the set of supplier modules** and continue (descend toward the concept — the concept is downstream). - [No] → **find the set of backtrack modules** (this branch is a dead end; the concept is not below here, so back up). (ConceptLocation p.10)

What it's used for / why it matters. The algorithm turns the local/composite distinction into a guaranteed, *systematic* traversal: it ensures you (a) never stop too early — you only halt on a **local** match, the one place worth changing; and (b) never wander forever — a class whose composite responsibility lacks the concept is pruned and you backtrack. This is essentially a guided depth-first search of the CDG with a human oracle for the "is the concept here?" judgments. Following it (with the component marks) is what makes concept location reproducible and exam-gradeable rather than guesswork.

When & how it's applied. The search **alternates between expanding into suppliers and backtracking** until a module whose **local** functionality contains the concept is found. The **interactive tool** version splits responsibility along two swim lanes: the **Computer** finds candidate module sets (starting modules, suppliers, backtrack targets — the mechanical graph work); the **Programmer** judges whether the concept is implemented locally or in the composite (the semantic work a machine can't do). (ConceptLocation p.21) The UMLEditor trace below is this loop run to completion, including a deliberate wrong turn and backtrack.

Status of components (search marks)

What it is. The marks are a four-value *bookkeeping system* — a status label attached to every class during the search, recording what you have learned about it so far. They are the algorithm's memory: (ConceptLocation p.11)

Mark	Meaning
Blank	The class was never inspected and is not scheduled for inspection (you know nothing about it yet).
Propagating	Inspected; its composite responsibility contains the concept, so the search <i>propagates</i> into it / its suppliers (a live path worth descending).
Unchanged	Inspected; its composite responsibility does not contain the concept (a dead end for this path — it will not be changed).
Next	The class is scheduled for inspection (a candidate queued up, not yet examined).

What it's used for / why it matters. The marks make the traversal *systematic and finite*. They prevent the two ways a manual graph search goes wrong: re-inspecting classes you have already judged (wasted effort, possible loops) and losing track of which candidates still need looking at. The **Propagating vs. Unchanged** split is the recorded outcome of the composite-responsibility question — it is how you remember which branches are live and which are pruned. The **Next vs. Blank** split is your to-do queue versus the untouched remainder. Together they let anyone (or the interactive tool) pick up the search at any point and know exactly what to do next.

When & how it's applied. As you run the algorithm, a class starts **Blank**; when the algorithm queues it as a candidate it becomes **Next**; once you inspect it, it becomes **Propagating** (concept in its composite → keep going) or **Unchanged** (concept not in its composite → backtrack). In the UMLEditor trace, `NoteNode` is inspected, found not to contain the concept, and marked **Unchanged** (the wrong-way/backtrack step), while `ClassNode` ends the search by locally containing the concept.

Reading the dependency-search activity diagram precisely (ConceptLocation p.10)

What it is. The deck draws the algorithm as a UML activity diagram whose exact nodes, guards, and loops are exam material in their own right. The elements, in order: a filled **start node**; activity "**Find set of starting modules**"; activity "**Select one module**"; a first **decision diamond** annotated by the note "*Is the concept implemented in the module?*" with guard **[Yes]** flowing directly to the filled-circle **end node** and guard **[No]** flowing to a second decision diamond; the second diamond annotated "*Is the concept implemented in the composite responsibility?*" with guard **[Yes]** flowing to activity "**Find set of the supplier modules**" and guard **[No]** flowing to activity "**Find set of backtrack modules**"; and a separately labelled transition **[Stop the search]** into the end node. (ConceptLocation p.10)

The loops matter. Both "Find set of the supplier modules" and "Find set of backtrack modules" flow back into "**Select one module**" — that double loop-back is what makes the search iterative: after every expansion *and* after every backtrack you are again selecting one module from the current candidate set and re-asking the two questions. There is exactly one way out of the loop: a [Yes] on the *first* question (concept implemented in the module itself), which is why the stop condition is always *local* implementation, never composite. (ConceptLocation p.10)

Question order is load-bearing. The local question is asked *before* the composite question on every iteration. Reversing them breaks the algorithm: a class whose local functionality contains the concept also has it in its composite functionality (the composite includes the module itself plus its suppliers, ConceptLocation p.9), so asking composite-first would always answer Yes and send you pointlessly into suppliers past the very class you should change. Exam answers that swap the diamonds lose the algorithm's termination property at the located class.

When & how it's applied. The interactive-tool slide (ConceptLocation p.21) repartitions this *same* diagram into swim lanes without changing any node: the three "find set of ..." activities sit in the **Computer** lane, while "Select one module" and both judgment diamonds sit in the **Programmer** lane. Memorize the diagram once and you can answer both the "draw the algorithm" question and the "what can be automated in concept location" question — the lane assignment is the answer to the latter. (ConceptLocation p.10, p.21)

Worked dependency-search trace — Rajlich's UMLEditor (Ch.6)

What it is and why it's here. This is the canonical *fully worked example* of the dependency-search algorithm: the deck runs the entire loop — marks and all — over one concrete Class Dependency Graph so you can see local-vs-composite decisions, supplier descent, a wrong turn, a backtrack, and a stop, in sequence. Its

purpose is to be the template you reproduce in the exam, so memorize the *shape of the walk* (which decision happens at each node), not just the class names.

The deck walks the algorithm over a CDG rooted at **UMLEditor** (a change request about *figure properties*), with children `SequenceDiagramGraph`, `UseCaseDiagramGraph`, `ClassDiagramGraph`, `ObjectDiagramGraph`, `StateDiagramGraph` and lower-level nodes `ClassRelationshipEdge`, `PackageNode`, `ClassNode`, `InterfaceNode`, `NoteNode`, `MultiLineString`, `RectangularNode`, `AbstractNode` (with `RectangularNode` $\triangleleft$ `AbstractNode` generalization and `ClassNode` $\triangle$ `RectangularNode`). (ConceptLocation p.12) The sequence: (ConceptLocation p.12–20)

1. **Start** at UMLEditor (root highlighted). (p.12)
2. **Classes to inspect** — UMLEditor's direct suppliers (the five `*DiagramGraph` classes) are marked Next. (p.13)
3. **Most likely supplier** — `ClassDiagramGraph` is chosen as the most promising supplier (bold arrow). (p.14)
4. **Next classes to inspect** — `ClassDiagramGraph`'s suppliers become Next. (p.15)
5. **Wrong way** — following toward `NoteNode` is a mistake (the concept is not there). (p.16)
6. **Backtrack** — `NoteNode` is marked Unchanged (grey) and the search backs up. (p.17)
7. **Concept location found** — `ClassNode` contains the concept (green). (p.18)
8. **Possible extension of the search** — search may continue into `RectangularNode` / `AbstractNode` (e.g., to widen the impact). (p.19)
9. **Another location found** — `AbstractNode` reached as a further location. (p.20)

This trace is the canonical exam example of dependency search *with* a wrong turn and backtrack — be ready to reproduce the Next/Propagating/Unchanged marking and the supplier/backtrack decision. (ConceptLocation p.12–20)

Impact Analysis (previewed)

What it is. Impact Analysis is the second *design* phase. Where concept location finds the *one* place to start, Impact Analysis *determines the strategy and the full impact of the change* — i.e., the complete set of classes that the change will touch or ripple into. (SoftwareChange p.9)

What it's used for / why it matters. A change rarely stays in a single class: editing one class can force changes in everything that depends on it. Impact Analysis exists to make that blast radius *visible before you start coding*, so you can plan, estimate effort and risk, and decide a strategy (and where prefactoring should localize the damage). Skipping it is how maintainers get surprised mid-change by cascading breakage.

When & how it's applied. It begins from concept location: the **classes identified in concept location make up the initial impact set**; then **class dependencies are analyzed and impacted classes are added to the impact set** by following the CDG outward. (SoftwareChange p.9) The slide visualizes this as a **"Concept" at the centre with an "Impact Set" radiating outward** through the software. (SoftwareChange p.7) For the watermark, the initial set (export action, drawing container, format writers) would grow to include any class a watermark step depends on. This is the direct **bridge from Lo2's concept location** to the next phase — and the reason the concept-location output is only the *initial* impact set, never the final one.

Prefactoring (previewed)

What it is. Prefactoring is *"opportunistic refactoring that localizes (minimizes) the impact of SC on the software."* (SoftwareChange p.10) It is *refactoring* (structure change, behavior unchanged) applied **before** the new functionality is written, expressly to reshape the code so the upcoming change will be small and contained. Two named Fowler refactorings are given as examples: (SoftwareChange p.10, [Fowler99]) - **Extract Class** — gather related fields, methods, and code snippets out of an overloaded class into a new, focused component class. - **Extract Superclass** — pull common members up into a new abstract class so variants can share them.

What it's used for / why it matters. Prefactoring is the process's lever for *minimizing impact*. If Impact Analysis shows a change would be sprawling — because the relevant logic is tangled into a big class or duplicated across siblings — you first tidy that structure so the actual change lands in one well-defined spot. It pays off as a smaller, lower-risk Actualization and easier Verification. Because it is *opportunistic*, you do it only where it demonstrably helps the change at hand, not as open-ended cleanup.

When & how it's applied. It happens *before* the new code is written. For the watermark, if every format writer duplicates "render the drawing," you might **Extract Superclass** to give them a shared base where the watermark step can be inserted once — turning a change-in-N-places into a change-in-one-place. This is exactly the "low-impact change" that JHotDraw's pattern-based design already tends to enable (see *How changes are made to JHotDraw*).

Actualization (previewed)

What it is. Actualization is the core *implementation* phase: it *creates the new code, plugs it into the old code, and visits neighboring classes to update them* so the whole stays consistent. (SoftwareChange p.11) This is the moment the change actually happens — new behavior is written and wired in.

What it's used for / why it matters. Plugging new code into an existing system is the step where consistency is most at risk, which is why the slide names the two phenomena that govern it. **Change propagation** is the *deliberate* act of updating each neighboring class so the system stays consistent after you alter one class. The **ripple effect** is the *consequence you must manage* — a change in one class cascading outward through dependencies, possibly forcing further changes. Understanding both tells you that Actualization is not "edit one class" but "edit one class *and chase the consequences* until the system is consistent again." This is also why Prefactoring (to shrink the ripple) and Verification (to catch propagation you missed) bracket it.

When & how it's applied. For the watermark you write the watermark-stamping code and plug it into the export path (e.g., a Decorator on figures, or a step in the shared format-writer base). Then you propagate: any class that calls or is called by the changed code is visited and updated, following the impact set from Impact Analysis, until the ripple settles. (SoftwareChange p.11)

Postfactoring (previewed)

What it is. Postfactoring is refactoring applied *after* the change has been actualized, to *eliminate any anti-patterns that the change may have introduced*. (SoftwareChange p.12) Like prefactoring it changes structure without changing behavior — but its target is the mess the *new* code just created. Examples of such anti-patterns: (SoftwareChange p.12) - **Long method** — after adding functionality, a method may now be doing too much and should be split. - **Bloated class** — after adding functionality, a class may have grown too large and should be decomposed.

What it's used for / why it matters. Adding functionality tends to *degrade* design: methods grow, classes accumulate responsibilities, and yesterday's clean structure becomes tomorrow's tangle. Postfactoring is the process's mechanism for **paying down the technical debt the change incurred**, so the codebase is left at least as maintainable as it was found — which keeps *future* changes cheap. Without it, a sequence of changes slowly rots the design (the link back to **preventive** maintenance). Crucially it is done *after* Verification has confirmed the new behavior, and is itself re-verified, since refactoring must preserve behavior.

When & how it's applied. It is the **mirror of prefactoring**: prefactor *before* to minimize impact, postfactor *after* to remove introduced anti-patterns. If wiring in the watermark left `export()` a 200-line method, you would extract the watermark logic into its own well-named method or class so the next maintainer isn't punished for your addition.

Verification (previewed)

What it is. **Verification** is the phase that *guarantees the correctness of the change*. Uniquely among the phases it is not a single step but a *continuous* activity that runs **vertically** across the implementation phases (Prefactoring → Conclusion). (SoftwareChange p.13) It comprises **testing** in three flavors and human review: (SoftwareChange p.13) - **functional testing** — does the change do what the request asked, from the outside? - **unit testing** — does each individual class/method behave correctly in isolation? - **structural testing** — are the internal paths/branches of the code exercised and correct? - **walkthroughs** — humans reading and reasoning through the change to catch what tests miss.

What it's used for / why it matters. Verification is what makes the whole change process *trustworthy*: it is the evidence that the new behavior is right *and* that nothing previously working was broken. It is drawn vertically precisely because correctness cannot be bolted on at the end — each refactoring and each propagation step needs checking as it happens, so an error is caught at its source rather than after it has rippled. The three test types give *complementary* coverage (external behavior, internal units, code paths), and walkthroughs cover reasoning errors that tests cannot encode.

When & how it's applied. Refactorings (pre/post) are verified by *existing* tests passing unchanged (proving behavior was preserved); the new watermark behavior is verified by *new* functional and unit tests plus structural coverage of the export paths. This is the course's hook into software testing and BDD covered in later lectures.

Conclusion (previewed)

What it is. **Conclusion** is the final phase and the second *interaction with the world*. It *commits the finished, verified code into version control, builds the new baseline, and prepares for the next change*. (SoftwareChange p.14) "Baseline" means the new committed, buildable state that becomes the agreed starting point for whatever change comes next.

What it's used for / why it matters. Conclusion is what turns a *finished* change into a *durable, shared* one. Committing to version control makes the change permanent, attributable, and recoverable; building the new baseline ensures the integrated system actually compiles and runs as a whole; and "prepare for the next change" closes the loop back to **Initiation**, so the eight phases form a repeatable cycle rather than a one-shot. Without a clean Conclusion the work is not really done — it isn't reproducible or buildable for the next maintainer.

When & how it's applied. In the labs this is the `git commit` / pull-request step under **GitHub flow**, producing the baseline the next user story builds on. It is the explicit tie-in to **Version Control / Git and CI**

covered later in the course. (SoftwareChange p.14)

JHotDraw — the framework (architecture)

What it is. JHotDraw is a **Java framework for drawing technical and structured graphics** such as network layouts and Gantt diagrams. (JHotDraw p.4) A *framework* (as opposed to a library) is a partially complete application that you complete by plugging in your own classes — it calls your code, not the other way around. JHotDraw was **originally developed in Smalltalk as HotDraw by Kent Beck and Ward Cunningham**, and was "*one of the first software development projects explicitly designed for reuse and labelled a framework.*" (JHotDraw p.4) The demo GUI shows tools, figures (ellipses, rectangles), connections, groups, text, annotations, images, and URL attachments. (JHotDraw p.4, p.6)

What it's used for / why it matters. In this course JHotDraw is the **running case study** — the real, non-trivial, pattern-dense codebase you fork and change. It matters because it is *deliberately reuse-oriented*: its architecture is small, well-named, and pattern-aligned, which makes it an ideal teaching specimen for concept location (domain concepts map cleanly to classes), impact analysis (dependencies are explicit), and pre/postfactoring. Knowing this core class model "cold" is what lets you orient inside the codebase quickly during the labs and the exam.

When & how it's applied — the core class model (the architecture students must know): (JHotDraw p.5, p.7) Each class below is both a thing you must name *and* a place a change might land:

- **DrawApplication** (*extends AWT Frame*) — the application/window; owns the current **Tool** and a **Drawing**, and is associated 1-1 with a **StandardDrawingView**. *It is the top-level entry point and a common starting class for concept location and for factory-method customization.* (JHotDraw p.5)
- **StandardDrawingView** (*extends AWT Panel*) — the **view**; holds the **selection** and receives **notification** from the Drawing when the model changes. *Where you'd look for rendering/selection concerns.* (JHotDraw p.5)
- **Drawing** (*interface*) — the **figure container**; aggregates 1..n **Figures** (black-diamond composition). *The model root that holds what gets exported/drawn.* (JHotDraw p.5)
- **Figure** (*interface*) — a drawable element; a Figure has 1..n **Handles** and Figures can themselves be figure containers (1 self-association) — the self-association is exactly what enables the **Composite** pattern. (JHotDraw p.5)
- **Tool** (*interface*) — the **controller**; the application's current tool (1). *Because "features start from controller classes," Tools are the usual concept-location entry points.* (JHotDraw p.5)
- **Handle** (*interface*) — interaction points on a figure (resize/move grips). *Where direct-manipulation behavior lives.* (JHotDraw p.5)

Extension points (how applications reuse the framework): application-specific classes subclass the framework interfaces — **MyTool / YourTool** extend **Tool**, and **MyFigure / YourFigure** extend **Figure**. (JHotDraw p.7) This is the *frozen spots vs. hot spots* idea: the framework's skeleton (the fixed collaboration among the classes above) is the *frozen spot*; the *hot spots* are the Tool/Figure subclasses you supply. This is the structural reason adding a new figure or tool is a low-impact change — you extend at the hot spots without editing the frozen skeleton.

Design patterns — the GoF foundation

What it is. A **design pattern** is a named, reusable solution to a recurring design problem — a template for how to arrange classes/objects, not a piece of code. The deck grounds JHotDraw in the **Gang-of-Four (GoF)**

catalogue [GHJV94], the 23 classic object-oriented patterns. (JHotDraw p.16) Beneath the catalogue sit three **principles** (themed *flexibility & reuse through decoupling*): (JHotDraw p.9)

- **Program to an interface, not to an implementation** — depend on abstractions so concrete classes can be swapped freely.
- **Favor composition over class inheritance** — assemble behavior from collaborating objects rather than rigid subclass hierarchies.
- **Find what varies and encapsulate it** — isolate the parts that change behind a stable interface so change stays local.

What it's used for / why it matters. Patterns are this course's link between *design quality* and *maintainability*. The three principles all push the same way: **decouple the stable from the variable**, so that a future change touches only the encapsulated, varying part. That is exactly the property the change process wants — small, localized impact. The catalogue gives a shared vocabulary ("this is a Strategy") that lets you recognize *where* a framework expects to be extended, which is invaluable during concept location and impact analysis.

When & how it's applied — the 23 patterns fall into **three categories** by *purpose*, each further split by *scope* (Class vs. Object): (JHotDraw p.10–11)

- **Creational** — patterns for *initializing and configuring* classes and objects (controlling *how things get made*): Factory Method (*class*); Abstract Factory, Builder, Prototype, Singleton (*object*). (JHotDraw p.12)
- **Structural** — patterns for *decoupling interface from implementation* (controlling *how things are composed*): Adapter (*class/object*); Bridge, Composite, Decorator, Facade, Flyweight, Proxy (*object*). (JHotDraw p.13)
- **Behavioral** — patterns for *dynamic interactions among objects* (controlling *how responsibilities and control flow are distributed*): Interpreter, Template Method (*class*); Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Visitor (*object*). (JHotDraw p.14)

(The *scope* axis: **Class**-scope patterns rely on inheritance fixed at compile time; **Object**-scope patterns rely on object composition that can vary at run time.)

General observation: extensibility of frameworks calls for extensive use of design patterns. (JHotDraw p.16) This single sentence is the lecture's thesis linking patterns to maintainability/changeability — and the reason a maintenance course spends so long on a pattern catalogue.

Patterns used in JHotDraw (the eight that matter)

What it is. This is the short list of the GoF patterns the framework actually relies on. JHotDraw makes extensive use of: **Model-View-Controller, Composite, Strategy, State, Template Method, Decorator, Factory Method, Prototype.** (JHotDraw p.16) (MVC is strictly a compound *architectural* pattern, not one of the original 23, but is listed here as JHotDraw's overall structure.)

What it's used for / why it matters. This list is the exam's high-value target and the practical map of *where JHotDraw is designed to be changed*. Each pattern marks an extension point: Composite for figure trees, Strategy for layouts, State for tool modes, Template Method for connections, Decorator for embellishments, Factory Method for menus/tools, Prototype for figure creation. Memorizing **intent + JHotDraw role + one pro/con** per pattern lets you both answer pattern questions and reason about which kind of change is low-impact.

When & how it's applied. Each is detailed, with its JHotDraw usage and consequences, in the *JHotDraw Connection* section below, and the *How changes are made to JHotDraw* subsection ties each pattern to a specific kind of change request.

The recurring process sidebar — color legend and how to read it

What it is. The phased-model figure that recurs on nearly every slide is not decoration; it encodes the course's taxonomy of the eight elements in color. The *Phased model of SC* slide pairs the chevron stack with an explicit legend: **orange = "Interactions with the world"**, **white = "SC design"**, **green = "SC Implementation"**. (SoftwareChange p.5) Reading the stack top-to-bottom with that legend: **Initiation** (orange), **Concept Location** (white), **Impact Analysis** (white), **Prefactoring** (green), **Actualization** (green), **Postfactoring** (green), **Conclusion** (orange), with the **VERIFICATION** box drawn as a vertical rectangle overlapping Prefactoring, Actualization, Postfactoring, and Conclusion. (SoftwareChange p.5)

Why it matters. The legend is the visual proof of the world-interaction / design / implementation grouping that exam questions ask you to reproduce, and the vertical VERIFICATION box is the visual proof that verification is *not* an eighth sequential step but a parallel activity spanning the implementation phases. (SoftwareChange p.5, p.13)

When & how it's applied. On every subsequent slide the single phase being taught is re-highlighted in orange/yellow against an otherwise plain stack — Initiation on p.6, Concept Location and Impact Analysis *jointly* in yellow on the "Design" slide (p.7), Concept Location alone on p.8, Impact Analysis on p.9, Prefactoring on p.10, Actualization on p.11, Postfactoring on p.12, the VERIFICATION box itself on p.13, and Conclusion on p.14. (SoftwareChange p.6–14) The *Change Initiation* deck opens by reusing the identical Initiation-highlighted figure (ChangeInitiation p.2), and the *Concept Location* deck opens with the Concept-Location-highlighted figure (ConceptLocation p.2) — the sidebar is the connective tissue across all three process decks.

The Design-phase figure — Concept, Impact Set, Software (SoftwareChange p.7)

What it is. The slide titled simply "*Design*" highlights Concept Location and Impact Analysis together (both yellow in the sidebar) and carries the lecture's single most quoted diagram: a large dark oval labelled **Software**, containing a jagged lighter region labelled **Impact Set**, with a small white rectangle labelled **Concept** at its centre. A stick-figure programmer stands *outside* the software with an arrow labelled **Concept Location** pointing inward at the Concept; arrows labelled **Impact Analysis** radiate *outward* from the concept into the jagged impact-set region. (SoftwareChange p.7)

Why it matters. The picture encodes the directionality of the two design phases: concept location is a *pinpointing* movement from outside the code to one spot inside it, while impact analysis is an *expanding* movement from that spot outward along dependencies. The jagged boundary of the impact set is deliberate — the affected region is irregular, following dependency structure rather than any tidy module boundary. The fact that one slide hosts both phases underlines their hand-off: the concept found is the seed of the impact set. (SoftwareChange p.7–9)

When & how it's applied. Be able to redraw and label this figure: Software (outer oval), Impact Set (jagged region), Concept (centre box), the inward Concept Location arrow from the programmer, and the outward Impact Analysis arrows. It is the canonical visual answer to "explain the relationship between concept location and impact analysis."

Dynamic program analysis — the foundation of the CLLab

What it is. The Concept Location Lab handout opens with a precise definition: *"Dynamic program analysis is the analysis of computer software that is performed by executing programs on a real or virtual processor."* (CLLab) It immediately adds the effectiveness condition: *"For dynamic program analysis to be effective, the target program must be executed with sufficient test inputs to produce interesting behavior. Use of software testing measures such as code coverage helps ensure that an adequate slice of the program's set of possible behaviors has been observed."* (CLLab)

Why it matters. This is the formal underpinning of the **dynamic search (execution traces)** methodology from the lecture (ConceptLocation p.7): what you observe at runtime is only as informative as the inputs that drove the execution. If you never trigger the feature path, the debugger never shows you its classes — the precise weakness attributed to dynamic search in the methodology comparison. **Code coverage** is named as the guard against concluding from an unrepresentative run: it measures how much of the possible behavior your inputs actually exercised. (CLLab)

When & how it's applied. In the lab you run JHotDraw under the **IDE debugger**, exercise your selected feature with inputs that actually produce the interesting behavior, and observe which classes execute — starting from the **controller classes**, because *"features are often started from the controller classes."* (CLLab) The lab's scoping rule — if the feature involves a large number of classes, *"localize only the domain classes that are related to the domain concepts"* — is the practical embodiment of partial comprehension's as-needed strategy. (CLLab, ConceptLocation p.3)

The UMLEditor trace with mark colors, step by step (ConceptLocation p.12–20)

What it is. The nine trace slides color-code every class using the marks table's own palette: **Blank** = **white/uncolored**, **Propagating** = **orange**, **Unchanged** = **grey**, **Next** = **green**. (ConceptLocation p.11) Reading the slides with that key gives the exact mark assignment at every step:

Sli de	Title	Mark/color state
p. 12	<i>Locating Figure Properties: Start</i>	UMLEditor green (Next) — scheduled as the starting module; every other class white (Blank).
p. 13	<i>Classes to inspect</i>	UMLEditor orange (Propagating) — inspected, composite responsibility contains the concept; its five suppliers (SequenceDiagramGraph , UseCaseDiagramGraph , ClassDiagramGraph , ObjectDiagramGraph , StateDiagramGraph) green (Next) .
p. 14	<i>Most likely supplier</i>	Bold purple arrow UMLEditor → ClassDiagramGraph ; ClassDiagramGraph stays green (the chosen candidate); the four sibling graphs drop to light grey (set aside, not chosen for inspection).
p. 15	<i>Next classes to inspect</i>	ClassDiagramGraph orange (Propagating) ; its five suppliers (ClassRelationshipEdge , PackageNode , ClassNode , InterfaceNode , NoteNode) green (Next) ; a bold blue arrow marks the path followed so far.
p. 16	<i>Wrong way</i>	Bold purple arrow ClassDiagramGraph → NoteNode — inspecting NoteNode is the deliberate mistake.
p. 17	<i>Backtrack</i>	NoteNode dark grey (Unchanged) — its composite responsibility does not contain the concept; the search backs up.

Slide	Title	Mark/color state
p. 18	<i>Concept location found</i>	<code>ClassNode</code> highlighted green at the end of the bold blue path <code>UMLEditor → ClassDiagramGraph → ClassNode</code> .
p. 19	<i>Possible extension of the search</i>	<code>ClassNode</code> now orange ; bold arrow <code>ClassNode → RectangularNode</code> (orange); <code>AbstractNode</code> green (Next) .
p. 20	<i>Another location found</i>	<code>AbstractNode</code> highlighted red — a second change location, reached via the inheritance chain (<code>ClassNode</code> specializes <code>RectangularNode</code> , which generalizes to <code>AbstractNode</code>).

Provenance. Every trace slide carries the attribution "© 2012 Václav Rajlich — *Software Engineering: The Current Practice, Ch. 6*" with the original Rajlich chapter-slide numbers 31–39 — these nine slides are imported verbatim from the textbook's own slide set, which is why the exam expects this exact example. (ConceptLocation p.12–20)

Two reading cautions. First, green does double duty: it is the *Next* mark throughout, but on p.18 the same green is used as the "found" highlight on `ClassNode` (and the *second* location on p.20 is highlighted red instead). Second, note that the extension steps (p.19–20) follow **inheritance (generalization) edges**, not only use/dependency edges — `RectangularNode < AbstractNode` with `ClassNode Δ RectangularNode` — so dependency search legitimately traverses the class hierarchy when the concept's implementation may be inherited. (ConceptLocation p.12, p.19–20)

The end-to-end Initiation diagram (ChangeInitiation p.8)

What it is. The final slide of the *Change Initiation* deck compresses the whole front half of the change process into one box-and-arrow picture: **Stakeholders** (drawn as two stick figures) —*Change Initiation*→ **Product Backlog** (solid box) which contains the **Change Request** (drawn as a *dashed* box inside the backlog) —*Change*→ **New Code** (small solid box) sitting on top of **Existing Code** (large solid box), with a *dashed* box labelled **Desired Code** enclosing the New Code. (ChangeInitiation p.8)

Why it matters. The dashing is semantic. The Change Request is dashed because it is a *statement of intent* held in the backlog, not yet realized; Desired Code is dashed because it is the *future state* the change aims at. The single arrow labelled "Change" between backlog and code is the entire eight-phase process (Concept Location through Conclusion) compressed to one edge — this slide is the bridge from the Initiation deck back to the phased model. Existing Code being the large, solid base box restates the course premise: change always lands *on top of* code that already exists. (ChangeInitiation p.8, SoftwareChange p.2)

When & how it's applied. Reproduce-and-label is the exam task: stakeholders, the Change Initiation arrow, Product Backlog containing the dashed Change Request, the Change arrow, New Code plugged atop Existing Code, and the dashed Desired Code envelope. In the lab workflow the concrete realizations are: stakeholders = you picking a JHotDraw feature; backlog = the GitHub Projects TODO column; change request = the user-story card; change = the work of later labs. (ChangeReqLab)

JHotDraw Connection

This lecture **introduces JHotDraw** as the course's running case study; everything below is the rich treatment of the framework and its patterns. JHotDraw is the codebase students fork, write user stories

against, and run concept location on. (ChangeReqLab, CLLab)

Why JHotDraw for a maintenance course

What it is / why it matters. JHotDraw is *designed for reuse* and *pattern-dense*, which makes it the ideal specimen for studying **change**. The reasoning is that maintenance techniques are easiest to *teach and see* on code that was built to be changed: in a well-factored, pattern-based framework, domain concepts map onto recognizable classes (clean concept location), dependencies are explicit (tractable impact analysis), and the seams for change are obvious (natural pre/postfactoring). (JHotDraw p.4, p.16) A messy, accidental codebase would obscure the very lessons the course wants to highlight.

When & how it's applied. Its history (Smalltalk HotDraw by Beck & Cunningham → Java JHotDraw) also makes it a canonical "first framework," so the patterns you learn here are the foundational ones. (JHotDraw p.4) Across the course you fork JHotDraw, write user stories against its existing features, run concept location on it with the IDE debugger, and carry changes through the eight phases — making it the connective tissue between the abstract process and concrete practice.

The demo GUI — what the screenshot teaches (JHotDraw p.4, p.6)

What it is. The same JHotDraw screenshot appears twice in the deck — once raw (p.4) and once annotated with the framework's three core vocabulary arrows (p.6): **Tools** points at the toolbar row of buttons under the menu bar, **Figures** points at the drawn shapes on the canvas, and **Drawing** points at the canvas area itself. The window shows the application titled *JHotDraw*, a menu bar reading **File, Edit, Align, Attributes, Debug, Animation, Images, Window**, an inner document window titled `C:\JHotDraw\drawings\demo.draw`, and a status bar reading **"Selection Tool"** — the name of the currently active tool. (JHotDraw p.4, p.6)

What the demo drawing contains. The sample document is itself a catalogue of feature labels: **Connections** (two ellipses joined by a line), **Groups** (stacked rectangles), **Text** (a text figure), **Annotation** and **Connected Text** (text attached to figures), **Images** (the embedded graphics, including the Java mascot), and **URL Attachments**. (JHotDraw p.4, p.6)

Why it matters / how it's applied. Two practical uses. First, the annotated screenshot is the fastest concrete answer to "map the GUI onto the architecture": toolbar buttons instantiate `Tool` objects (controller), the shapes are `Figure`s held by the `Drawing` (model), and what you see through the document window is the `DrawingView` (view) — with the status bar displaying which Tool is *current*, mirroring `DrawApplication`'s `current tool 1` association. (JHotDraw p.5–6, p.17) Second, the labelled features (connections, groups, text, annotation, images, URL attachments) are exactly the kind of **existing features** the ChangeReqLab asks you to pick from when writing your user story — the screenshot is effectively the feature menu for the portfolio assignment. (ChangeReqLab, JHotDraw p.6)

Framework boundary — JHotDraw classes vs. application-specific classes (JHotDraw p.5, p.7)

What it is. The two architecture slides draw an explicit ownership boundary. On p.5 the diagram is partitioned into **"Java AWT classes"** (`Frame`, `Panel` at the top) and **"JHotDraw classes"** (everything below them). On p.7 the *Using the Framework* slide adds a third tier labelled **"application-specific classes"**: `MyTool` and `YourTool` specializing `Tool`, `MyFigure` and `YourFigure` specializing `Figure`. (JHotDraw p.5, p.7)

Why it matters. The three tiers define who owns what during a change. AWT classes are the platform — never yours to change. JHotDraw classes are the *framework skeleton* — the fixed collaboration

(DrawApplication↔StandardDrawingView↔Drawing↔Figure↔Tool↔Handle with its cardinalities) that a well-designed change should leave untouched. Application-specific classes are *yours* — the designated landing zone for new behavior. The deliberate extension points are precisely the two interfaces given application-specific subclasses on the slide: **Tool** and **Figure**. (JHotDraw p.7) This is also why the framework's *interfaces* (*Drawing*, *Figure*, *Tool*, *Handle* are drawn in italics — abstract) matter so much: "program to an interface, not to an implementation" (JHotDraw p.9) is what lets the framework call *your* *MyFigure* through the *Figure* interface without knowing it exists.

When & how it's applied. During impact analysis on any JHotDraw change request, classify each candidate class by tier: if the impact set starts reaching *upward* into framework classes rather than *downward* into your application-specific subclasses, that is the signal to prefactor — or to reconsider the design — because the framework expected the change to land at its hot spots (Tool/Figure subclasses, factory-method overrides, new strategies/states/decorators), not in its skeleton. (JHotDraw p.7, p.16, p.43; SoftwareChange p.10)

Architecture recap (classes to know cold)

DrawApplication (Frame) ▶ owns ▶ Tool (current tool) and ▶ Drawing; paired 1-1 with StandardDrawingView (Panel) which holds the **selection** and gets **notification**. Drawing (figure container) aggregates Figures 1..n; each Figure owns Handles 1..n and may itself contain figures. Applications subclass Tool and Figure (MyTool, MyFigure, ...). (JHotDraw p.5, p.7) Memorize the cardinalities: DrawApplication 1-1 StandardDrawingView, Drawing 1-1..n Figure, Figure 1-1..n Handle. (JHotDraw p.5)

Model-View-Controller (MVC) in JHotDraw

What it is. MVC is an architectural pattern that splits an interactive application into three responsibilities so they can change independently: the **Model** (the data and rules), the **View** (the presentation), and the **Controller** (input handling). JHotDraw's overall structure is MVC: (JHotDraw p.17)

- **Model** — Figures and their attributes (FillColor, Position) + Drawing (the figure container). *The "what is drawn."*
- **View** — DrawingView (a clipping view of a window) + DrawingWindow. *The "how it looks on screen."*
- **Controller** — Tools that manipulate the model. *The "what the user does."*

What it's used for / why it matters. MVC's payoff is **decoupling**: because the model knows nothing about the view, you can change rendering without touching the data, add a second view of the same drawing, or swap the input tools without disturbing either. For a maintenance course this is the headline example of how separation of concerns *localizes change* — a presentation change stays in the view, an interaction change stays in a tool. It also tells you, during concept location, *which third of the system* a concept lives in (data → model classes, look → view classes, behavior on input → tool classes).

When & how it's applied. The interaction loop: the View raises a **User Action** → the Controller handles it → it **Updates** the Model → the Model **Notifies** observers → the View **Updates** to reflect the new state. (JHotDraw p.17) The class-level picture: Figure and Drawing are <<model>> interfaces (with AbstractFigure, StandardDrawing implementations; FigureAttributes holds an fMap:Hashtable of attribute values); DrawingView is the <<view>> interface (StandardDrawingView); Tool is the <<controller>> interface (AbstractTool), exposing mouse methods mouseDown/Drag/Up/Move and lifecycle activate/deactivate. (JHotDraw p.18) The watermark concept, being about data on export, lives on the **model** side; a new editing gesture would live in a **controller** (Tool).

Composite in JHotDraw

What it is (intent). Composite is a *structural* pattern that **composes objects into tree structures for part-whole hierarchies, letting clients treat individual objects and compositions uniformly.**

(JHotDraw p.19) The key trick is that the "whole" and the "part" share one interface, so client code cannot tell — and does not care — whether it holds a single object or a tree of them. **Structure:** a `Component` interface (`Operation / Add / Remove / GetChild`) with `Leaf` and `Composite` subclasses; the `Composite` holds children and implements `Operation()` by forwarding to each child (`forall g in children: g.Operation()`).

(JHotDraw p.20–21)

What it's used for / why it matters. Composite is what lets you build arbitrarily nested structures while keeping client code dead simple — no special cases for "group vs. single." For maintenance this is doubly valuable: clients don't change when you add a new leaf or composite kind (open for extension), and operations defined once on the interface automatically work on whole trees. The cost is the type-safety weakness noted below: because everything is just a `Component`, the type system can't stop you from putting the wrong child into a composite.

When & how it's applied. In JHotDraw a **figure is composed of several figures**; the composite figure and its child figures are treated uniformly, so `scale`, `rotation`, and `move` exert common behavior on all of them at once. (JHotDraw p.22) Concretely: `Figure` (abstract, `draw():void`) is subclassed by `RectangleFigure`, `LineFigure`, ..., and by `GraphicalCompositeFigure`, which **delegates its methods to all contained Shapes**. (JHotDraw p.23) This allows **nested structures of arbitrary depth** — a group inside a group inside a drawing. (JHotDraw p.23) **Pros:** wherever client code expects a primitive it also accepts a composite; simpler clients; easy to add new component kinds without changing clients. **Cons:** can make the design overly general; you **can't use the type system to restrict** which components a composite holds — you must fall back to run-time checks. (JHotDraw p.24) *Change tie-in:* adding a new figure type means subclassing `Figure` and slotting it into this hierarchy — a textbook low-impact, open-for-extension change.

Strategy in JHotDraw

What it is (intent). Strategy is a *behavioral* pattern that **defines a family of algorithms, encapsulates each one, and makes them interchangeable** so the algorithm can vary independently of the clients that use it. (JHotDraw p.25) Instead of hard-coding one algorithm (or branching among several with conditionals), the client holds a reference to a `Strategy` object and calls it polymorphically. **Use when** many related classes differ only in behavior, you need several variants of an algorithm, an algorithm uses data clients shouldn't see, or a class has accumulated many behaviors as `if/switch` conditionals. (JHotDraw p.26) **Structure:** a `Context` holds a `strategy: Strategy`; the `Strategy` interface declares `AlgorithmInterface()`; `ConcreteStrategyA/B/C` each implement it differently. (JHotDraw p.27)

What it's used for / why it matters. Strategy turns "behavior" into a *pluggable, swappable object*, which is the direct embodiment of "find what varies and encapsulate it." Its maintenance value: you add a new behavior by writing a new strategy class — no editing of the context, no growing conditional. It also lets the *same* client pick among implementations with different time/space trade-offs at run time. The price is that the client must know which strategies exist in order to choose one.

When & how it's applied. In JHotDraw, **layout algorithms are separated from the objects being laid out** (mirroring Java AWT/Swing layout managers). Every **layouter** can be attached to a composite figure to arrange its children differently — e.g., *no layout*, *align-to-top*, and *align-to-left* produce different arrangements of the same set of figures. (JHotDraw p.28) **Pros:** families of related algorithms; an alternative to subclassing; eliminates conditional statements; a choice of implementations with different time/space

trade-offs. **Cons:** clients must be aware of the different strategies; communication overhead between Strategy and Context; an increased number of objects. (JHotDraw p.29) *Change tie-in:* adding a new layout is just a new Strategy (layouter) — figures themselves never change.

State in JHotDraw

What it is (intent). State is a *behavioral* pattern that **lets an object alter its behavior when its internal state changes — so much that the object appears to change its class.** (JHotDraw p.30) Each distinct state is reified as its own object implementing a common `State` interface; the context delegates to whichever state object is current, and behavior changes simply by swapping that object. **Use when** an object's behavior depends on its state and must change at run-time, or when operations have large, multipart, state-dependent conditionals. (JHotDraw p.30) **Structure:** a `Context` holds a `state: State`; its `Request()` delegates to `state->Handle()`; `ConcreteStateA/B` each implement `Handle()` for their state. (JHotDraw p.31)

What it's used for / why it matters. State replaces a sprawling state machine encoded as nested conditionals with a clean set of small classes, one per state. The maintenance win: a new mode/state is a **new subclass**, not an edit to a giant conditional, and the legal transitions become explicit in code rather than implicit in tangled `if` s. It is essentially Strategy where the "algorithm" is the object's mode and the object itself decides when to switch.

When & how it's applied. In JHotDraw the goal is to **externalize a tool's state** so a single tool can operate in different **modes**. (JHotDraw p.32) Examples: a **zoom tool** toggling between "zoom in" and "zoom out"; a **selection tool** toggling between "select border" and "select text." The deck shows `SelectTool` holding a `SelectToolState` with `ZeroClickState`, `OneClickState`, `TwoClickState` — e.g., in a text field, **1 click sets the cursor, 2 clicks select a word, 3 clicks select the whole line**, each handled by a different state object. (JHotDraw p.32) **Pros:** it localizes state-specific behavior and partitions behavior across states; new states and transitions are added just by defining new subclasses; it makes state transitions explicit; and state objects can be shared. (JHotDraw p.33) *Change tie-in:* adding a new tool mode is a new State subclass — no touching of existing conditionals.

Template Method in JHotDraw

What it is (intent). Template Method is a *behavioral* pattern that **defines the skeleton of an algorithm in one operation, deferring some steps to subclasses** so they can redefine those steps without changing the algorithm's overall structure. (JHotDraw p.34) The base class owns the *invariant* control flow and calls out to overridable "hook" methods for the *variant* steps. (It is a *class-scope* pattern — the variation point is fixed by inheritance.)

What it's used for / why it matters. Template Method is the canonical way a *framework* enforces a fixed process while letting *applications* customize the details — the textbook "Hollywood principle: don't call us, we'll call you." Its maintenance value is that the shared algorithm lives in exactly one place (no duplication across variants), and adding a new variant means overriding a few hook methods rather than reimplementing or editing the whole algorithm. It is the inheritance-based counterpart to Strategy.

When & how it's applied. In JHotDraw the framework defines the *invariant* algorithm for **connecting figures via a line** in one class (`LineConnection`), exposing the *variant* parts as empty methods that subclasses overwrite. From the framework's point of view, "connecting figures with a line" is always the same operation; from the application's point of view it differs by connection semantics — e.g., **Petri-net connections** versus **class-diagram connections**. (JHotDraw p.35) Template Method joins these two views: one fixed skeleton, application-specific steps plugged in. *Change tie-in:* supporting a new connection semantics means overriding `LineConnection`'s variant hooks — a localized extension, not a rewrite.

Decorator in JHotDraw

What it is (intent). Decorator is a *structural* pattern that **attaches additional responsibilities to an object dynamically**, providing a flexible alternative to subclassing for extending functionality. (JHotDraw p.36) A decorator *wraps* a component, implements the same interface, forwards calls to the wrapped object, and adds its own behavior before/after — so decorators can be stacked at run time. **Use when** you want to add responsibilities to individual objects transparently (without affecting other objects), for responsibilities that can be *withdrawn*, or when extension by subclassing would cause a **subclass explosion** (too many combinations to enumerate as classes). (JHotDraw p.37) **Structure:** Component / ConcreteComponent ; a Decorator holds a component and forwards Operation() (component->Operation()); ConcreteDecoratorA/B add state/behavior around that call. (JHotDraw p.38)

What it's used for / why it matters. Decorator's strength is *composable, per-object, run-time* extension: instead of BorderedShadowedRectangle , BorderedRectangle , ShadowedRectangle ... you compose a plain figure with a border decorator and a shadow decorator as needed. For maintenance this avoids a combinatorial blow-up of subclasses and keeps the base classes lean. Its characteristic pitfall is *identity*: because the real object is hidden behind wrappers, type checks and reference equality stop being reliable.

When & how it's applied. In JHotDraw, Decorator adds **specific visualization to generic visualization** — e.g., add **borders and shadows** to a figure via a DecoratorFigure with BorderDecorator and ShadowDecorator . (JHotDraw p.39) **Caveat noted on the slide:** the type of a decorated object is hard to obtain if it isn't exposed via methods like getDecoratedType() (Java's instanceof operator won't see through the wrapper) and if no separate list of all figures (including the decorated ones) exists. (JHotDraw p.39) **Pros:** more flexibility than static inheritance; avoids feature-laden classes high up in the hierarchy. **Cons:** a decorator and its component aren't identical; lots of little objects. (JHotDraw p.40) *Change tie-in:* adding a new visual embellishment is a new Decorator, not a subclass — and a watermark could itself be implemented as a Decorator on figures.

Factory Method in JHotDraw

What it is (intent). Factory Method is a *creational* pattern that **defines an interface for creating an object but lets subclasses decide which class to instantiate** — it "lets a class defer instantiation to subclasses." (JHotDraw p.41) The base class calls a factoryMethod() wherever it needs a product, without naming a concrete class; subclasses override that method to choose the concrete product. **Use when** a class can't anticipate the class of objects it must create, wants its subclasses to specify the objects it creates, or wants to localize the knowledge of which helper subclass is the delegate. (JHotDraw p.41) **Structure:** a Creator whose AnOperation() calls FactoryMethod() (product = FactoryMethod()); a ConcreteCreator overrides FactoryMethod() to return new ConcreteProduct . (JHotDraw p.42)

What it's used for / why it matters. Factory Method removes hard-coded new ConcreteClass() calls from generic code, so the *framework* can drive object creation while *applications* decide what gets created. The maintenance value is that you customize *what* is built by overriding a method rather than editing the framework — a clean hook point and a way to keep parallel hierarchies (creators ↔ products) in step.

When & how it's applied. In JHotDraw, factory methods keep **menus and tools flexible**. DrawApplication defines createMenus() and createTools() ; a customized MyDrawApplication (e.g., ClassDiagramDrawApplication , FigureDrawApplication) inherits from it and **overrides** those methods to add its own tools (for class/interface figures, or for circles/boxes). (JHotDraw p.43) **Consequences:** it provides hooks for the subclasses, and it connects parallel class hierarchies. (JHotDraw p.44) *Change tie-in:* registering a new figure type into the app's menus/tools is done by overriding these factory methods — the framework's creation logic stays untouched.

Prototype in JHotDraw

What it is (intent). Prototype is a *creational* pattern that **specifies the kinds of objects to create using a prototypical instance, and creates new objects by copying (cloning) that prototype.** (JHotDraw p.45) Rather than instantiating a class by name, you keep a ready-made example object and call `clone()` on it to produce more. **Use when** the classes to instantiate are specified at run-time (e.g., dynamic loading), when you want to avoid building a factory hierarchy that parallels the product hierarchy, or when instances have only a few combinations of state — installing a few prototypes and cloning them is then more convenient than instantiating the class manually. (JHotDraw p.45–46) **Structure:** a `Prototype` interface with `clone()`; concrete prototypes return copies of themselves; a client creates products by cloning a prototype it was handed.

What it's used for / why it matters. Prototype decouples object creation from concrete classes *without* a parallel factory class for every product type — the "configured example" carries all the information needed to make more. Its maintenance value: you can add or remove product kinds at run time just by registering or removing prototypes, and clients work with application-specific classes without knowing their names. It pairs naturally with Composite (clone a whole sub-tree) and complements Factory Method (cloning instead of subclassing the creator).

When & how it's applied. In JHotDraw, **each tool is initialized with a prototype figure** — an instance of the figure it is meant to create — and when creating a new figure the tool simply **clones the prototype** (`prototype.clone()`). The Circle Tool clones a circle prototype; the Rectangle Tool clones a rectangle prototype. (JHotDraw p.46–47) **Consequences:** it hides the concrete product classes from clients (fewer names to know); it lets clients work with application-specific classes without modification; it lets you add and remove products at run-time; and it lets you specify new objects by varying prototype values. (JHotDraw p.48) *Change tie-in:* to make a new figure creatable, register a prototype for a tool to clone — no new tool class and no edits to existing creation code.

How changes are made to JHotDraw (tying patterns to maintenance)

The patterns are not decoration — they are the **change-enablers** this course exploits: - To add a **new figure type:** subclass `Figure` /use the Composite hierarchy; register a **prototype** so a tool can clone it; the **factory methods** wire it into menus/tools. (JHotDraw p.43, p.47, p.23) - To add a **new tool mode:** add a **State** subclass rather than editing conditionals. (JHotDraw p.32) - To add a **new layout:** add a **Strategy** (layouter) without touching figures. (JHotDraw p.28) - To add **new visual embellishments:** add a **Decorator**, not a subclass. (JHotDraw p.39) - To support a **new connection semantics:** override the variant hooks of the **Template Method** `LineConnection`. (JHotDraw p.35)

Each of these is a low-impact change — exactly the "minimize impact" goal of prefactoring and the changeability the framework was designed for. This is why concept location on JHotDraw is tractable: well-named, pattern-aligned classes map cleanly to domain concepts.

GoF Pattern Catalogue — Complete Intent Reference

The JHotDraw deck does not only name the eight patterns the framework uses — it enumerates the **whole GoF catalogue** across three slides, giving the verbatim *intent* of most of the 23 patterns. (JHotDraw p.10–14) The pattern sections elsewhere in this guide cover the eight that JHotDraw exploits; this section is the complete catalogue reference exactly as the slides give it, because exam questions can ask for any intent on these slides.

The Purpose × Scope classification table (JHotDraw p.11)

The deck's master table crosses two axes — **Purpose** (Creational / Structural / Behavioral) and **Scope** (Class / Object): (JHotDraw p.11)

Scope	Creational	Structural	Behavioral
Classes	Factory Method	Adapter (class)	Interpreter, Template Method
Object	Abstract Factory, Builder, Prototype, Singleton	Adapter (object), Bridge, Composite, Decorator, Facade, Flyweight, Proxy	Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Visitor

Reading notes: **Adapter is the only pattern that appears in both scopes** (it has a class-inheritance form and an object-composition form). Counting the table: 5 creational + 7 structural + 11 behavioral = 23, the canonical GoF count, with class-scope patterns being the small minority (Factory Method, Adapter-class, Interpreter, Template Method). Class-scope patterns fix their variation point at compile time via inheritance; object-scope patterns vary at run time via composition. (JHotDraw p.10–11)

Creational patterns — all five intents verbatim (JHotDraw p.12)

Creational patterns *"deal with initializing and configuring classes and objects."* (JHotDraw p.10)

- **Factory Method** (class scope) — *"Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory method lets a class defer instantiation to subclasses."* (JHotDraw p.12)
- **Abstract Factory** (object) — *"Provide an interface for creating families of related or dependent objects without specifying their concrete class."* (JHotDraw p.12)
- **Builder** (object) — *"Separate the construction of a complex object from its representation so that the same construction process can create different representations."* (JHotDraw p.12)
- **Prototype** (object) — *"Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype."* (JHotDraw p.12)
- **Singleton** (object) — *"Ensure a class only has one instance, and provide a global point of access to it."* (JHotDraw p.12)

Of the five, JHotDraw uses **Factory Method** (createMenus/createTools) and **Prototype** (tools cloning prototype figures). (JHotDraw p.16, p.43, p.47)

Structural patterns — all seven intents verbatim (JHotDraw p.13)

Structural patterns *"deal with decoupling interface and implementation of classes and objects."* (JHotDraw p.10)

- **Adapter** (class/object) — *"Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces."* (JHotDraw p.13)
- **Bridge** (object) — *"Decouple an abstraction from its implementation so that the two can vary independently."* (JHotDraw p.13)
- **Composite** (object) — *"Compose objects into tree structures to represent whole-part hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly."* (JHotDraw p.13)
- **Decorator** (object) — *"Attach additional responsibilities to an object dynamically."* (JHotDraw p.13)
- **Facade** (object) — *"Provide a unified interface to a set of interfaces in a subsystem."* (JHotDraw p.13)

- **Flyweight** (object) — *"Use sharing to support large numbers of fine-grained objects efficiently."* (JHotDraw p.13)
- **Proxy** (object) — *"Provide a surrogate or placeholder for another object to control access to it."* (JHotDraw p.13)

Of the seven, JHotDraw uses **Composite** (figure trees) and **Decorator** (border/shadow embellishments). (JHotDraw p.16, p.22–23, p.39)

Behavioral patterns — the six intents the deck spells out (JHotDraw p.14)

Behavioral patterns *"deal with dynamic interactions among societies of classes and objects."* (JHotDraw p.10)

- **Interpreter** (class) — *"Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language."* (JHotDraw p.14)
- **Template Method** (class) — *"Define the skeleton of an algorithm in an operation, deferring some steps to subclasses; lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure."* (JHotDraw p.14)
- **Chain of Responsibility** (object) — *"Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it."* (JHotDraw p.14)
- **Command** (object) — *"Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations."* (JHotDraw p.14)
- **Iterator** (object) — *"Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation."* (JHotDraw p.14)
- **Mediator** (object) — *"Define an object that encapsulates how a set of objects interact."* (JHotDraw p.14)

Completeness note: the Purpose × Scope table (p.11) also lists **Memento**, **Observer**, **State**, **Strategy**, **Visitor** as object-scope behavioral patterns, but the enumeration slide stops after Mediator — **State's and Strategy's intents are instead given on their dedicated JHotDraw slides** (*State*: "Allow an object to alter its behavior when its internal state changes. The object will appear to change its class." p.30; *Strategy*: "Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it." p.25), while Memento, Observer, and Visitor get no intent statement anywhere in this deck. (JHotDraw p.11, p.14, p.25, p.30) Of the eleven behavioral patterns, JHotDraw uses **Strategy**, **State**, and **Template Method**. (JHotDraw p.16)

Citation key on the slide: [GHJV95]

The *Design Patterns in JHotDraw* slide cites the catalogue as *"Design Patterns [GHJV95]"* — Gamma, Helm, Johnson, Vlissides. (JHotDraw p.16) Elsewhere this guide keys the same Gang-of-Four book as [GHJV94] (the commonly used 1994 imprint key); both keys denote the identical reference, so do not be thrown if an exam source uses either year.

Memorization scaffolding — counting and grouping the 23

Useful checks derived directly from the Purpose × Scope table (JHotDraw p.11):

- **Per-purpose counts:** 5 creational, 7 structural, 11 behavioral — behavioral is the biggest family by far (it covers "dynamic interactions among societies of classes and objects," JHotDraw p.10).
- **Class-scope is rare:** only **Factory Method** (creational), **Adapter-class** (structural), and **Interpreter** + **Template Method** (behavioral) are class-scope; every other pattern works by object composition. That

asymmetry is itself the second GoF principle in action — "favor composition over class inheritance." (JHotDraw p.9, p.11)

- **The only dual-scope entry is Adapter**, listed once under Class and once under Object. (JHotDraw p.11)
- **JHotDraw's picks per category**: creational 2 of 5 (Factory Method, Prototype), structural 2 of 7 (Composite, Decorator), behavioral 3 of 11 (Strategy, State, Template Method), plus the MVC compound that is not one of the 23. So of the 23, JHotDraw's deck deep-dives exactly 7, and MVC makes the published list of 8. (JHotDraw p.16)
- **Intent coverage check**: the deck states verbatim intents for all 5 creational (p.12), all 7 structural (p.13), and 6 behavioral patterns (p.14), plus Strategy (p.25), State (p.30), and Template Method again (p.34) on their feature slides — leaving exactly **Memento, Observer, and Visitor** with no intent text anywhere in the deck. If an exam question asks for an intent "from the slides," those three are the trick options. (JHotDraw p.11–14, p.25, p.30, p.34)

Pattern Mechanics — UML Structures and API Surface from the Slides

The deck pairs every JHotDraw pattern with a GoF structure diagram and (usually) a JHotDraw class diagram. The class names, method signatures, and diagram annotations below are exam-grade detail — they are exactly what appears on the slides.

MVC class diagram — full API surface (JHotDraw p.18)

The *Model View Controller (2)* slide gives the framework's interfaces with their operations:

- **Figure** — `<<model>>` interface: `+moveBy:void`, `+center:void`, `+size:Dimension`, `+clone:Object`. Implemented by **AbstractFigure**, which has an `attributes` association to **FigureAttributes** (`<<model>>`), whose members are `-fMap:Hashtable` (private), `+set:void`, `+get:Object`. (JHotDraw p.18)
- **Drawing** — `<<model>>` interface: `+add:Figure`, `+remove:Figure`, `+figures:FigureEnumeration`, `+findFigure:Figure`. Implemented by **StandardDrawing**. **Figure** is aggregated into **Drawing** (open-diamond aggregation on the slide). (JHotDraw p.18)
- **DrawingView** — `<<view>>` interface: `+activate:void`, `+deactivate:void`, `+mouseDown:void`, `+mouseDrag:void`, `+mouseUp:void`, `+mouseMove:void`. Implemented by **StandardDrawingView**; holds a drawing reference to **Drawing**. (JHotDraw p.18)
- **Tool** — `<<controller>>` interface: the same six operations (`+activate:void`, `+deactivate:void`, `+mouseDown:void`, `+mouseDrag:void`, `+mouseUp:void`, `+mouseMove:void`). Implemented by **AbstractTool**; holds a drawing reference to **Drawing**. (JHotDraw p.18)

Two exam-worthy observations: `+clone:Object` on the **Figure** interface is literally the **Prototype** hook in the model API — the operation a tool calls on its prototype figure (JHotDraw p.18, p.47); and the mouse/lifecycle methods on **Tool** are where the **State** pattern's delegation happens when a tool subdivides its modes (JHotDraw p.18, p.32).

Composite — structure and object diagrams (JHotDraw p.20–23)

GoF class structure (p.20). **Client** points at the abstract **Component**, which declares `Operation()`, `Add(Component)`, `Remove(Component)`, `GetChild(int)`. **Leaf** implements only `Operation()`. **Composite** implements all four and holds a `children` aggregation back to **Component**; its `Operation()` carries the slide's

annotation `forall g in children: g.Operation();` — the one-line essence of the pattern (the whole forwards the operation to every part). (JHotDraw p.20)

Object structure (p.21). The instance diagram shows an `aComposite` root with three `aLeaf` children plus one nested `aComposite` that itself holds three `aLeaf`s — the canonical picture of nesting at arbitrary depth. (JHotDraw p.21)

JHotDraw realization (p.22–23). A worked composite figure of **5 subordinate figures** is shown being *composed*, then *scaled and rotated* as a single unit — operations exert common behavior on all members. (JHotDraw p.22) The class diagram has abstract `Figure (draw():void)` with subclasses `RectangleFigure`, `LineFigure`, "...", and `GraphicalCompositeFigure`, which holds a `shape` aggregation back to `Figure` and carries the note "*delegate methods to all contained Shapes.*" (JHotDraw p.23)

Strategy — structure diagram (JHotDraw p.27–28)

`Context` holds a `strategy` aggregation (open diamond) to the abstract `Strategy`; `Context` exposes `ContextInterface()`, `Strategy` declares `AlgorithmInterface()`, and `ConcreteStrategyA`, `ConcreteStrategyB`, `ConcreteStrategyC` each override `AlgorithmInterface()`. (JHotDraw p.27) The JHotDraw slide explicitly parallels **Java AWT/Swing layout managers** — "every layouter (Java Swing / AWT: layout manager) can be attached to a composite figure rendering it" — and demonstrates with three renderings of the same three circles: *no layout manager*, *"align to top"*, *"align to left."* (JHotDraw p.28)

State — structure diagram (JHotDraw p.31–32)

GoF structure (p.31). `Context` with `Request()` holds a `state` aggregation to abstract `State` with `Handle()`; the annotation on `Request()` reads `state->Handle()`. `ConcreteStateA` / `ConcreteStateB` each implement `Handle()`. (JHotDraw p.31)

JHotDraw realization (p.32). `SelectTool (+select:void)` holds a `state` reference to `SelectToolState (+select:void)` with the note "*calls state.select()*"; the concrete states are `ZeroClickState`, `OneClickState`, `TwoClickState` (each `+select:void`). The worked example is a text field: *set the cursor with 1 click, select a word with 2 clicks, select the whole line with 3 clicks.* (JHotDraw p.32) Naming trap: the GoF structure calls the state operation `Handle()`, but JHotDraw's realization calls it `select()` — same role, different name.

Decorator — object chain and class structure (JHotDraw p.36–39)

GoF object diagram (p.36). The classic chain: `aBorderDecorator` — `component` → `aScrollDecorator` — `component` → `aTextView` — a text view wrapped in a scroll decorator wrapped in a border decorator, rendered on the slide as the resulting bordered, scrollable text window. (JHotDraw p.36)

GoF class structure (p.38). Abstract `Component (Operation())` is specialized by `ConcreteComponent` and by abstract `Decorator`, which holds a `component` aggregation back to `Component`; `Decorator.Operation()` carries the note `component->Operation()` (pure forwarding). `ConcreteDecoratorA` adds a field `addedState`; `ConcreteDecoratorB` adds `AddedBehavior()` with the note `Decorator::Operation(); AddedBehavior();` — forward first, then add behavior. (JHotDraw p.38)

JHotDraw realization (p.39). Interface `Figure` is implemented by `DecoratorFigure` (aggregating exactly 1 `Figure`), specialized by `BorderDecorator` and `ShadowDecorator`; the picture sequence shows a text+image figure plain → with border → with border and shadow. (JHotDraw p.39)

Factory Method — structure and JHotDraw hierarchy (JHotDraw p.42–43)

GoF structure (p.42). **Product** $\triangleleft$ **ConcreteProduct**; **Creator** declares `FactoryMethod()` and `AnOperation()` with the note `product = FactoryMethod()` (the framework calls its own factory method); **ConcreteCreator** overrides `FactoryMethod()` with the note `return new ConcreteProduct` and a dashed creates-relationship to `ConcreteProduct`. (JHotDraw p.42)

JHotDraw realization (p.43). **DrawApplication** (labelled *Framework class*) declares the factory methods as *protected*: `#createMenus:void`, `#createTools:void`. Two customized application classes inherit and override them as *public*: **ClassDiagrammDrawApplication** (`+createMenus:void`, `+createTools:void`; note: "Add tools for class and interface figures" — the Class Diagram Application) and **FigureDrawApplication** (note: "Add tools for circles, boxes, etc." — the Figure Application). (JHotDraw p.43) Spelling quirk: the slide really does write `ClassDiagrammDrawApplication` with a double *m*.

Prototype — structure as JHotDraw usage (JHotDraw p.46–47)

Uniquely, the deck's Prototype "structure" slide is already the JHotDraw realization: the **Circle Tool** holds a circle **Prototype** instance and `prototype.clone()` yields a stream of circle figures; the **Rectangle Tool** holds a rectangle **Prototype** and `prototype.clone()` yields rectangles. (JHotDraw p.46) The accompanying text slide restates the rule: "Each tool is initialized with an instance (a prototype) of the figure it is meant to create. When creating a new figure, the tool clones the prototype." (JHotDraw p.47)

Diagram-notation glossary used across the L02 slides

The decks assume fluent UML reading; these are the notations actually used, with where each appears:

- **Black (filled) diamond — composition.** `DrawApplication`'s ownership of its `Tool`, its `Drawing`, and its pairing toward `StandardDrawingView`; `Drawing`'s containment of `Figures`. The owned part's lifetime is bound to the whole. (JHotDraw p.5, p.7)
- **Open (hollow) diamond — aggregation.** `Context`→`Strategy` (`strategy`), `Context`→`State` (`state`), `Decorator`→`Component` (`component`), `Composite`→`Component` (`children`), `Drawing` aggregating `Figure` on the MVC class diagram, `DecoratorFigure` aggregating `1 Figure`, `GraphicalCompositeFigure`'s `shape` link. Looser whole-part than composition — parts can outlive or be swapped. (JHotDraw p.18, p.20, p.23, p.27, p.31, p.38–39)
- **Hollow triangle on a solid line — generalization (inheritance).** `Frame` $\triangleleft$ `DrawApplication`, `Panel` $\triangleleft$ `StandardDrawingView` (JHotDraw p.5); `RectangularNode` $\triangleleft$ `AbstractNode` with `ClassNode` $\triangle$ `RectangularNode` in the UMLEditor CDG (ConceptLocation p.12); `Component` $\triangleleft$ `Leaf/Composite` (JHotDraw p.20); all the `ConcreteStrategy/ConcreteState/ConcreteDecorator/ConcreteCreator` subclassings. (JHotDraw p.27, p.31, p.38, p.42)
- **Hollow triangle on a dashed line — interface realization.** `AbstractFigure` realizes `Figure`, `StandardDrawing` realizes `Drawing`, `StandardDrawingView` realizes `DrawingView`, `AbstractTool` realizes `Tool`. (JHotDraw p.18)
- **Dashed arrows — dependency.** The edges of the Class Dependency Graph itself (UMLEditor depends on its `DiagramGraphs`, etc.) and the `ConcreteCreator`-creates-`ConcreteProduct` link. (ConceptLocation p.12, JHotDraw p.42)
- **Italics — abstract.** `Drawing`, `Figure`, `Tool`, `Handle` on the architecture diagram; `Component`, `Strategy`, `State`, `Decorator`, `Creator`, `Product` in the GoF structures. Abstract elements are exactly where applications plug in. (JHotDraw p.5, p.20, p.27, p.31, p.38, p.42)
- **Visibility markers.** `+` `public` (the interface operations on p.18 and the overridden factory methods), `#` `protected` (`#createMenus`, `#createTools` in `DrawApplication`), `-` `private` (`-fMap:Hashtable` in

FigureAttributes). (JHotDraw p.18, p.43)

- **Stereotypes in guillemets.** `<<model>>`, `<<view>>`, `<<controller>>` tagging the MVC roles of the interfaces; `<<name>>`, `<<intension>>`, `<<extensions>>` tagging the concept-triangle example. (JHotDraw p.18, ConceptLocation p.5)
 - **UML notes (dog-eared boxes with dashed anchors).** Carry the executable essence of patterns: `forall g in children: g.Operation();` (Composite), `state->Handle()` and `"calls state.select()"` (State), `component->Operation()` and `Decorator::Operation(); AddedBehavior();` (Decorator), `product = FactoryMethod()` and `return new ConcreteProduct` (Factory Method), `"delegate methods to all contained Shapes"` (JHotDraw Composite), and the two question-notes on the dependency-search diamonds. (JHotDraw p.20, p.23, p.31–32, p.38, p.42; ConceptLocation p.10)
 - **Activity-diagram elements.** Filled start node, rounded activities, decision diamonds with bracketed guards ([Yes], [No], [Stop the search]), the bullseye end node, and — on the interactive-tool version — Computer/Programmer swim lanes. (ConceptLocation p.10, p.21)
 - **Cardinalities.** `1`, `1..n` adorning the architecture associations (Drawing 1—1..n Figure, Figure 1—1..n Handle, DrawApplication 1—1 StandardDrawingView, current tool 1). (JHotDraw p.5)
-

Model Exam Answers (Fully Written Out)

Five high-frequency exam prompts with complete, slide-grounded answers in the register an examiner expects. Use these as templates — every claim cites its slide.

Model answer 1 — "Describe the phased model of software change."

Software change (SC) is the process of adding new functionality to existing code and is the foundation of software evolution and servicing. (SoftwareChange p.2) The course's phased model decomposes one SC into seven sequential phases plus one spanning activity. (SoftwareChange p.5) **Initiation** starts the change: a change request arrives — a software bug, an enhancement, or an improvement — and change requests are prioritized. (SoftwareChange p.6) **Concept Location** extracts concepts from the change request and locates them in the code as the starting point of the SC. (SoftwareChange p.8) **Impact Analysis** determines the strategy and impact of the change: the classes identified in concept location make up the initial impact set, then class dependencies are analyzed and impacted classes are added to the impact set. (SoftwareChange p.9) **Prefactoring** is opportunistic refactoring that localizes (minimizes) the impact of the SC on the software — e.g., Fowler's Extract Class (gathering fields, methods, and code snippets into a new component class) or Extract Superclass (creating a new abstract class). (SoftwareChange p.10) **Actualization** creates the new code, plugs it into the old code, and visits neighboring classes to update them — managing change propagation and the ripple effect. (SoftwareChange p.11) **Postfactoring** eliminates anti-patterns the change may have introduced, such as a long method (a method now doing too much) or a bloated class (a class grown too large). (SoftwareChange p.12) **Conclusion** commits the finished code into version control, builds the new baseline, and prepares for the next change. (SoftwareChange p.14) **Verification** is not a sequential phase: it runs vertically alongside Prefactoring through Conclusion and guarantees the correctness of the change via functional, unit, and structural testing plus walkthroughs. (SoftwareChange p.5, p.13) The phases group into *interactions with the world* (Initiation, Conclusion), *SC design* (Concept Location, Impact Analysis), and *SC implementation* (Prefactoring, Actualization, Postfactoring). (SoftwareChange p.5)

Model answer 2 — "Execute dependency search with marks on the UMLEditor example."

Dependency search is a static concept-location technique that walks a Class Dependency Graph (CDG) extracted from the existing code, deciding at each class between two notions of functionality: *local*

functionality (concepts actually implemented in the module, not delegated to others) and *composite* functionality (the module's complete functionality combined with all its supporting modules), both determined by reading code and documentation. (ConceptLocation p.9) During the search, every class carries one of four marks: Blank (never inspected, not scheduled), Propagating (inspected; composite responsibility contains the concept), Unchanged (inspected; composite responsibility does not contain the concept), Next (scheduled for inspection). (ConceptLocation p.11)

On the UMLEditor CDG (a change to *figure properties*): the search starts at the root, `UMLEditor`, scheduled Next. (ConceptLocation p.12) Inspecting it shows the concept in its composite — but not local — functionality, so `UMLEditor` becomes Propagating and its five suppliers (`SequenceDiagramGraph`, `UseCaseDiagramGraph`, `ClassDiagramGraph`, `ObjectDiagramGraph`, `StateDiagramGraph`) become Next. (ConceptLocation p.13) The programmer selects the most likely supplier, `ClassDiagramGraph`. (ConceptLocation p.14) Inspection marks it Propagating and schedules its suppliers (`ClassRelationshipEdge`, `PackageNode`, `ClassNode`, `InterfaceNode`, `NoteNode`) as Next. (ConceptLocation p.15) The programmer then follows a wrong way into `NoteNode` (ConceptLocation p.16); inspection finds the concept absent from its composite responsibility, so `NoteNode` is marked Unchanged and the search backtracks. (ConceptLocation p.17) Re-selecting among the Next candidates, the programmer inspects `ClassNode` and finds the concept implemented locally — concept location found; the search may stop. (ConceptLocation p.18) Optionally the search extends along the inheritance chain through `RectangularNode` toward `AbstractNode` (ConceptLocation p.19), where another location is found — figure properties are also implemented in the superclass. (ConceptLocation p.20) The stop rule throughout: stop only when the concept is implemented in the module itself; composite-only containment means descend to suppliers; absence from the composite means backtrack. (ConceptLocation p.10)

Model answer 3 — "Apply the concept triangle to the watermark change request."

A concept comprises three linked elements: its **Name** (the label), its **Intension** (its definition — connected to the name by the *naming/definition* edge), and its **Extension** (the set of its instances — connected to the intension by *recognition/location* and to the name by *annotation/traceability*). (ConceptLocation p.4) The deck's illustration: the name "Dog / Pes / Hund" (one concept, three language-specific names), the intension "Hairy animal with teeth...", and the extension {Fido, Lassie, Buck from *Call of the Wild*, ...}. (ConceptLocation p.5)

Applied to the request "*Modify the export feature of JHotDraw to automatically include a simple watermark text in the drawings being exported ... uniformly for all possible export formats*", concept extraction yields **Export, Drawing, Text, Format**. (ConceptLocation p.6) For each concept, the triangle warns that the *name* in the request need not match the *name* in the code — exactly as "Dog", "Pes", and "Hund" name one concept. The reliable search edge is therefore Intension → Extension: hold the definition of *Export* ("turning a drawing into an external file format") and *recognize* the code that satisfies it, whatever it is called; the located classes are the concept's extension in the code. (ConceptLocation p.4–6) This is why pure name-based search (GREP — "global regular expression print", printing lines that match a regular expression, used iteratively, ConceptLocation p.8) can only seed the search, while dependency search and dynamic search, which judge what code *does* rather than what it is *called*, can finish it. (ConceptLocation p.7–9)

Model answer 4 — "Explain MVC in JHotDraw, including the interaction loop."

JHotDraw's overall structure is the Model-View-Controller pattern. (JHotDraw p.16–17) The **Model** is the drawing data: `Figure`s with their attributes (`FillColor`, `Position`) and the `Drawing`, which is the figure container. The **View** is the presentation: `DrawingView` (a clipping view of a window) and `DrawingWindow`. The **Controller** is the input side: `Tool`s that manipulate the model. (JHotDraw p.17) The interaction loop runs:

the View raises a *User Action* to the Controller; the Controller *Updates* the Model; the Model *Notifies*; and the View *Updates* to show the new state — the View never edits the Model directly. (JHotDraw p.17)

At class level, `Figure` and `Drawing` are <<model>> interfaces — `Figure` declaring `moveBy`, `center`, `size`, `clone`, implemented by `AbstractFigure` whose attributes live in `FigureAttributes` (a private `fMap:Hashtable` with `set/get`); `Drawing` declaring `add`, `remove`, `figures` (returning a `FigureEnumeration`), and `findFigure`, implemented by `StandardDrawing`. `DrawingView` is the <<view>> interface (implemented by `StandardDrawingView`) and `Tool` the <<controller>> interface (implemented by `AbstractTool`); both declare `activate`, `deactivate`, and the four mouse operations `mouseDown`, `mouseDrag`, `mouseUp`, `mouseMove`. (JHotDraw p.18) The maintenance payoff is decoupling: a rendering change stays in view classes, an interaction change stays in tools, and a data change stays in figures/drawing — MVC tells you during concept location which third of the system a concept inhabits. (JHotDraw p.17)

Model answer 5 — "Why does extensibility of frameworks call for extensive use of design patterns? Illustrate with three JHotDraw patterns."

The deck's general observation is exactly that sentence: *"Extensibility of frameworks calls for extensive use of design patterns."* (JHotDraw p.16) The mechanism is the GoF principles the patterns implement — program to an interface, not an implementation; favor composition over class inheritance; find what varies and encapsulate it (JHotDraw p.9) — which together ensure that an application can extend the framework at designated points without editing the framework's own code.

Three illustrations. **Factory Method:** `DrawApplication` calls its own `createMenus()` and `createTools()`; a customized application like `ClassDiagramDrawApplication` or `FigureDrawApplication` inherits and overrides them to add its tools — the framework drives creation, the application decides what is created, and no framework class is edited. (JHotDraw p.43) **Strategy:** layout algorithms are separated from the objects laid out; any layouter (the analogue of a Swing/AWT layout manager) can be attached to a composite figure, so a new arrangement — align-to-top, align-to-left — is a new strategy class, not a modification of figures. (JHotDraw p.28) **Template Method:** the invariant algorithm for connecting figures with a line lives once in `LineConnection`, with variant steps exposed as empty methods; supporting a new connection semantics — Petri-net versus class-diagram connections — means overriding those hooks in a subclass. (JHotDraw p.35) In each case the pattern converts what would be an invasive framework edit into a *low-impact incremental change* in application-specific code — precisely the property the change process's prefactoring phase tries to create when it is missing ("localize/minimize the impact of SC", *SoftwareChange* p.10), and the reason a maintenance course teaches a pattern catalogue at all. (JHotDraw p.16)

Worked Example / Process Walkthrough

Putting Lo2 together on the JHotDraw watermark request:

1. Initiation. A stakeholder (here a manager wanting demo/trial parity with competitors) submits a request. (ChangeInitiation p.3) It is logged in the **Product Backlog** and prioritized (likely "Nice to have"). (ChangeInitiation p.7)

2. Change Request as a User Story. Rewrite the request to fit a 3"×5" card: (ChangeInitiation p.4–5, ConceptLocation p.6)

"As a trial-version user, I want exported drawings to automatically carry a watermark text so that the demo origin is visible in every export format." If it doesn't fit, split it (e.g., one story per export format).

3. Concept Location — extract concepts. Underline the domain concepts in the request: **Export, Drawing, Text, Format.** (ConceptLocation p.6) Reason with the **concept triangle**: the *intension* ("text stamped onto an exported drawing") guides recognition of code *extensions* (the classes that actually export). (ConceptLocation p.4)

4. Concept Location — locate in code. Choose a methodology: (ConceptLocation p.7) - **Dynamic search / IDE debugger** (the CLLab approach): run JHotDraw, trigger an export, and watch which classes execute. Features often start from **controller** classes, so begin at the export Tool/menu action. (CLLab) - **Static dependency search**: from the export starting module, ask "*is Export implemented here (local) or in the composite?*" If local → stop; if composite → descend into supplier modules; if neither → backtrack. Mark classes **Next / Propagating / Unchanged** as you go. (ConceptLocation p.10–11) - Optionally **GREP** for `export`, `write`, `format` to seed candidate files. (ConceptLocation p.8)

This mirrors the **UMLEditor trace**: pick the most likely supplier, follow it, recognize a *wrong way*, **backtrack**, and finally mark "concept location found" on the class that locally implements export. (ConceptLocation p.12–20)

5. Produce the initial class set (portfolio artifact). Record the located classes in the required table — the bridge into Impact Analysis: (CLLab)

Domain Class	Responsibility
(e.g.) <code>ExportAction</code> / export Tool	Controller entry point that initiates export
<code>Drawing</code> / <code>StandardDrawing</code>	Holds the figures to be exported
(<i>format writer class</i>)	Renders the drawing to a specific export format

6. Hand-off. These classes become the **initial impact set** for Impact Analysis; the watermark functionality (likely a **Decorator** on figures or a step in each format writer via **Template Method**) is then designed, prefactored to localize impact, actualized, postfactored, verified, and concluded by a commit + new baseline. (SoftwareChange p.9–14) — the remaining phases previewed but executed in later lectures.

Lab walkthrough (what the student actually does): - *ChangeReqLab*: fork the JHotDraw repo on GitHub, adopt **GitHub flow**, pick an existing feature, write its **user story**, and put a card in the TODO backlog via **GitHub Projects** — the user story is a portfolio artifact. (ChangeReqLab) - *CLLab*: use the **IDE debugger** to localize the classes involved in your change request (start from controller classes; if many classes, keep only domain classes), then write the **initial set of classes** in a *Domain Class | Responsibility* table — also a portfolio artifact. (CLLab)

The watermark request dissected sentence by sentence (ConceptLocation p.6)

The full verbatim change request — including the middle sentence usually skipped — is:

"Modify the export feature of JHowDraw to automatically include a simple watermark text in the drawings being exported. Similar functionality is commonly found, for instance, in trial or demo versions of applications. The watermark should be included uniformly for all possible export formats."
(ConceptLocation p.6)

Each sentence does distinct work. **Sentence 1** is the functional ask, and it carries the underlined domain concepts the slide extracts: *export* (the feature to modify), *text* (what the watermark is), *drawings* (what gets stamped) — three of the four extracted concepts in a single sentence. The word *automatically* is a requirement too: no user action should be needed per export. **Sentence 2** is pure motivation/context — "trial or demo versions of applications" — the *why* that a user story would put in its *so that* slot; it contributes no new concept but identifies the stakeholder (whoever ships trial versions) and supports prioritization. (ChangeInitiation p.5) **Sentence 3** is a *uniformity constraint*: the underlined *formats* yields the fourth concept and silently multiplies the scope — whatever the located solution is, it must apply across *every* export format, which is exactly the property that later argues for a single shared insertion point (a Template Method step in a common base, or a Decorator on the drawing) rather than per-format edits. (ConceptLocation p.6, JHotDraw p.35, p.39) **Concepts: Export, Drawing, Text, Format.** (ConceptLocation p.6)

A second end-to-end narration — the UMLEditor "figure properties" change through L02's phases

The watermark walkthrough above starts from a request and ends at located classes; the UMLEditor example can be narrated the same way to drill the phase vocabulary on the deck's *other* worked system:

- 1. Initiation.** A change request concerning **figure properties** of the UMLEditor arrives (the trace slides' own title: "*Locating Figure Properties*") and is recorded and prioritized like any request. (ConceptLocation p.12, SoftwareChange p.6)
- 2. Concept extraction.** The request's domain concept is *figure properties* — the thing to find in code. (ConceptLocation p.12)
- 3. Methodology choice.** With no expert and no traceability links assumed, the deck demonstrates **static dependency search** over the CDG extracted from the existing code. (ConceptLocation p.7, p.9)
- 4. The search itself.** Start at the client root `UMLEditor` (Next) → inspect: composite contains the concept, local does not → Propagating; five `*DiagramGraph` suppliers become Next → select the most likely supplier `ClassDiagramGraph` → inspect: Propagating; its five suppliers become Next → wrong way into `NoteNode` → Unchanged, backtrack → inspect `ClassNode`: concept implemented **locally** → stop, concept location found. (ConceptLocation p.12–18, p.10)
- 5. Optional widening.** Because figure properties may be partially implemented in superclasses, extend along `ClassNode` → `RectangularNode` → `AbstractNode` and record `AbstractNode` as another location. (ConceptLocation p.19–20)
- 6. Hand-off.** The located classes — `ClassNode`, plus `AbstractNode` from the extension — make up the **initial impact set**; Impact Analysis next analyzes class dependencies and adds impacted classes (plausibly `RectangularNode` between them, and the `ClassDiagramGraph` client above) before any code is touched. (SoftwareChange p.9)

The two narrations differ instructively: the watermark example exercises *concept extraction* richly (four concepts from prose) but the deck does not run its search; the UMLEditor example skips extraction (one named concept) but runs the *search* to completion with marks. Together they cover the whole of Concept Location — which is presumably why the deck includes both. (ConceptLocation p.6, p.12–20)

Lab Handouts in Full

Both LO2 labs are one-page handouts with an identical structure — *Introduction, Objectives, Classwork, Portfolio Work* — and both end in a named **portfolio artifact**. Everything below is the complete content of each handout.

ChangeReqLab — CASE Study Lab (full task breakdown)

Introduction (verbatim). *"Software changes are started by creating a change request. A change request can be a new feature, a bug fix and improvements. User stories are short and simple descriptions of features written from the perspective of the person who desires the new capability."* (ChangeReqLab) — note this restates the lecture's user-story definition almost word for word (ChangeInitiation p.5), substituting "features" for "capabilities."

Objectives (two). (1) To write a user story for an existing software feature (the handout's typo: "feature"). (2) Select a user story for *"your mandatory individual portfolio assignment."* (ChangeReqLab)

Classwork (five tasks, in order). (ChangeReqLab) 1. Each **Team** creates a **GitHub Fork** of the project repository — the handout links it as [JHotDraw] . 2. In future lab exercises, each team member follows **[GitHub flow]** to collaborate on projects — branching/PR discipline is set up now so later changes can be carried through Conclusion-style commits. 3. **Select an existing feature in JHotDraw and write the user story** — the handout points to a linked list of [existing features] , so the story is *reverse-engineered* from behavior that already exists rather than invented. 4. Use **[GitHub Projects]** — the course's backlog tool. 5. **Create a Card for your User Story and put it in the TODO Backlog** — the concrete realization of "add the change request to the Product Backlog." (ChangeInitiation p.7–8)

Portfolio Work. *"Write the User Story for your selected JHotDraw feature. Note this is an artifact for your portfolio."* (ChangeReqLab)

Reading notes. The bracketed names ([JHotDraw], [GitHub flow], [existing features], [GitHub Projects]) are hyperlink references in the original handout. The fork is per-*team* but the portfolio assignment is explicitly *individual* — each student writes and owns their own story. Writing a story for an *existing* feature is deliberate pedagogy: you must observe behavior and recover the *who/what/why*, exercising exactly the perspective-taking the user-story format demands. (ChangeReqLab, ChangeInitiation p.5)

CLLab — Concept Location Lab (full task breakdown)

Introduction (verbatim). *"Dynamic program analysis is the analysis of computer software that is performed by executing programs on a real or virtual processor. For dynamic program analysis to be effective, the target program must be executed with sufficient test inputs to produce interesting behavior. Use of software testing measures such as code coverage helps ensure that an adequate slice of the program's set of possible behaviors has been observed."* (CLLab)

Objectives (two). (1) Apply the **IDE Debugger** to locate feature concepts *at runtime*. (2) Create a list of the **initial set of classes** from the concept-location results. (CLLab)

Classwork (one composite task). Use the *IDE Debugger* to localize the classes that involve your *Change Request*. The handout gives two operational hints: *"Features are often started from the controller classes"* (in JHotDraw: the `Tool` hierarchy — the `<<controller>>` of the MVC structure, JHotDraw p.17–18), and *"if your feature involves a large number of classes then localize only the domain classes that are related to the domain concepts"* (the handout's duplicated-word typo: "to the to domain concepts"). (CLLab)

Portfolio Work. Write the initial set of classes **in table format**, with exactly two columns: **Domain Class** | **Responsibility**. (CLLab)

Reading notes. This lab operationalizes the **dynamic search** methodology from the lecture (ConceptLocation p.7) and its output — the initial class set — is precisely the **initial impact set** that Impact Analysis consumes next (SoftwareChange p.9). The "domain classes only" scoping rule is partial comprehension in action: effort proportional to the change, not the codebase. (ConceptLocation p.3)

How the two labs trace the lecture's process phases

Lab	Process phase exercised	Concrete tools	Portfolio artifact
ChangeReqLab (CASE Study Lab)	Initiation / Change Request	JHotDraw (forked repo), GitHub Fork, GitHub flow, GitHub Projects (TODO Backlog card)	The user story for a selected existing JHotDraw feature
CLLab (Concept Location Lab)	Concept Location	JHotDraw (running build), IDE Debugger (dynamic analysis), code-coverage thinking for input sufficiency	The initial set of classes as a Domain Class / Responsibility table

Together the two labs carry one feature through the first two phases of the canonical process, with GitHub flow already in place so that later phases (Actualization through Conclusion) can land as commits on the team fork. (ChangeReqLab, CLLab, SoftwareChange p.5)

Definitions & Terminology

Term	Definition	Source
Software Change (SC)	What: the disciplined process of adding new functionality to existing code; the foundation of evolution and servicing. Use: gives maintenance a traceable, localized, verifiable workflow instead of ad-hoc edits; every course activity is one trip through its eight phases.	SoftwareChange p.2
Perfective maintenance	What: change motivated by improving/extending functionality for users (e.g., adding credit-card support); ~50% of effort. Use: the dominant category — it justifies a process built around <i>adding</i> capability, and signals that new tests for new behavior are needed.	SoftwareChange p.3
Adaptive maintenance	What: change motivated by adapting software to a changed <i>external</i> environment — new OS, hardware, library, regulation, or data format (e.g., Y2K); ~25%. Use: flags compatibility work; what changes is the surroundings, not the user-visible feature.	SoftwareChange p.3
Corrective maintenance	What: change motivated by fixing bugs/defects where code fails its spec; ~21%. Use: the only "repair" category; typically driven by a failing-test-first approach.	SoftwareChange p.3
Preventive maintenance	What: change motivated by improving code structure to forestall future problems; ~4%. Use: pays down technical debt before it bites; usually realized <i>as</i> refactoring (the link between the why-axis and the how-axis).	SoftwareChange p.3
Incremental change	What: an impact-on-functionality kind that <i>adds</i> new behavior (the behavior set grows). Use: requires new tests; the watermark feature is incremental.	SoftwareChange p.4
Contraction	What: an impact kind that <i>removes</i> obsolete functionality (the behavior set shrinks). Use: requires checking nothing still depends on the removed feature.	SoftwareChange p.4
Replacement	What: an impact kind that <i>swaps</i> an existing feature for a different implementation/behavior. Use: retires old tests and adds new ones for the replacement.	SoftwareChange p.4

Term	Definition	Source
Refactoring	What: changing software <i>structure</i> without changing behavior (externally observable behavior is identical before/after). Use: the verification proof is that <i>existing</i> tests still pass; underpins both Prefactoring and Postfactoring.	SoftwareChange p.4
Initiation	What: the first phase (a world-interaction) where a change request arrives, is captured as requirements, and prioritized. Use: the funnel that prevents working on the wrong thing or in the wrong order; seeds traceability and the Lientz & Swanson classification.	SoftwareChange p.6; ChangeInitiation p.2
Change Request	What: the artifact produced by Initiation — the recorded requirement, as a sentence/paragraph, bug report, or user story, stored in the backlog. Use: the unit of work every later phase consumes; its form controls how much ambiguity survives into design.	ChangeInitiation p.4, p.8
User Story	What: a short, simple capability description from the desirer's perspective — "As a [user], I want [goal] so that [reason]" — that fits a 3"×5" card. Use: keeps the request in the problem space (reducing wrong-thing risk), names whom to validate with and the value to protect, and supplies the domain concepts for concept location.	ChangeInitiation p.4–5
Product Backlog	What: a single prioritized "wish list" of requirements (Must/Nice-to-have/Won't have) whose items are added/deleted/modified over time. Use: the team's shared queue and single source of truth for "what's next"; the container holding a Change Request between Initiation and the Change.	ChangeInitiation p.7
Concept Location	What: the design activity that finds the code snippet where a change is to be made, starting from the domain concepts in the request. Use: bridges the vocabulary gap between request words and code; produces the <i>starting point</i> (initial impact set) every later phase needs.	ConceptLocation p.2
Partial comprehension	What: achieving only the minimum essential understanding, via an as-needed strategy (analogy: "visiting a large city"). Use: the cognitive justification for concept location — keeps understanding effort proportional to the change instead of reading the whole system.	ConceptLocation p.3
Concept (triangle)	What: a concept modelled as Name + Intension (definition) + Extension (set of instances), linked by naming/definition, recognition/location, and annotation/traceability. Use: explains why name-only search fails and tells you to locate via Intension → Extension (recognize code by meaning, not label).	ConceptLocation p.4
Intension	What: the definition / defining properties of a concept (what makes something an instance). Use: the meaning you hold in mind to <i>recognize</i> matching code even when it's named differently.	ConceptLocation p.4
Extension	What: the set of all instances a concept denotes — in code, the actual classes/methods that implement it. Use: the target of the search; what you ultimately point to as the change location.	ConceptLocation p.4
GREP	What: "global regular expression print" — prints the lines matching a regex, treating code as text. Use: the quick, zero-setup static pattern-matching tool to <i>seed</i> candidate files iteratively; blind to dependencies and renaming, so paired with dependency search.	ConceptLocation p.8
Class Dependency Graph (CDG)	What: a graph (nodes = classes, edges = depends-on/uses) extracted from existing code. Use: the map dependency search walks — following edges from a start class toward the concept.	ConceptLocation p.9
Local functionality	What: the concepts a module implements <i>itself</i> and does not delegate to others. Use: the <i>stop</i> test — when the concept is in a class's local functionality, you've found the place to change.	ConceptLocation p.9
Composite functionality	What: a module's complete functionality combined with all its supplier modules (what it delivers <i>through everything it calls</i>). Use: the <i>descend</i> test — concept in the composite but not local ⇒ the work is downstream, so follow suppliers.	ConceptLocation p.9

Term	Definition	Source
Supplier module	What: a module that a selected module depends on. Use: searched next when the concept is in the composite (not local) responsibility — the algorithm's "descend" target.	ConceptLocation p.10
Backtrack	What: returning to a previous module when the current path's composite responsibility lacks the concept. Use: prunes dead-end branches so the search stays finite; an expected, normal step (see the UMLEditor NoteNode wrong-turn).	ConceptLocation p.10, p.17
Blank (mark)	What: a class never inspected and not scheduled for inspection. Use: the default/unknown status; the untouched remainder of the graph.	ConceptLocation p.11
Propagating (mark)	What: an inspected class whose composite responsibility <i>contains</i> the concept. Use: marks a live path the search descends into.	ConceptLocation p.11
Unchanged (mark)	What: an inspected class whose composite responsibility does <i>not</i> contain the concept. Use: marks a pruned dead end, triggering backtrack; it will not be changed.	ConceptLocation p.11
Next (mark)	What: a class scheduled (queued) for inspection. Use: the search's to-do list — distinct from Blank (Next is scheduled, Blank is neither inspected nor scheduled).	ConceptLocation p.11
Impact Analysis	What: the second design phase that determines the change strategy and the full impact set (initial impact set = the classes from concept location). Use: makes the change's blast radius visible before coding, enabling planning, risk estimation, and prefactoring.	SoftwareChange p.9
Impact Set	What: the set of classes affected by a change, grown outward from the concept via dependency analysis. Use: the scope of work; what Actualization must update and Verification must cover.	SoftwareChange p.7, p.9
Prefactoring	What: opportunistic refactoring <i>before</i> the change to localize/minimize its impact (e.g., Extract Class / Extract Superclass). Use: reshapes code so the upcoming change lands in one spot — a smaller, lower-risk Actualization.	SoftwareChange p.10
Actualization	What: the core implementation phase — create new code, plug it into the old code, and update neighbors. Use: where the change actually happens; managed via change propagation and the ripple effect to keep the system consistent.	SoftwareChange p.11
Change propagation	What: the deliberate act of updating neighboring classes so the system stays consistent after a change. Use: ensures the edit doesn't leave dependents broken; driven by the impact set.	SoftwareChange p.11
Ripple effect	What: changes cascading outward through dependencies, possibly forcing further changes. Use: the consequence you must chase during Actualization; prefactoring shrinks it, verification catches what it broke.	SoftwareChange p.11
Postfactoring	What: refactoring <i>after</i> the change to remove anti-patterns it introduced (long method, bloated class). Use: pays down the technical debt the change created so future changes stay cheap; the mirror of prefactoring.	SoftwareChange p.12
Verification	What: the phase guaranteeing correctness via testing (functional/unit/structural) and walkthroughs. Use: the trust mechanism; runs <i>vertically</i> across implementation phases so errors are caught at their source, not after rippling.	SoftwareChange p.13
Conclusion	What: the final world-interaction phase — commit verified code to version control, build the new baseline, prepare for the next change. Use: makes the change durable and shared; closes the loop back to Initiation (the Git/CI tie-in).	SoftwareChange p.14
JHotDraw	What: a Java <i>framework</i> for technical/structured graphics; descended from Smalltalk HotDraw (Beck & Cunningham); an early reuse-designed framework. Use: the course's running, pattern-dense case study — forked, written-against, and concept-located on in the labs.	JHotDraw p.4
DrawApplication	What: the framework's application/window class (extends AWT Frame); owns the current Tool and Drawing; paired 1—1 with StandardDrawingView. Use: the top-level entry point and a common concept-location start; customized via Factory Method (createMenus/createTools).	JHotDraw p.5

Term	Definition	Source
StandardDrawingView	What: the view (extends AWT Panel); holds the selection and receives notification from the Drawing. Use: where rendering/selection concerns live in the MVC structure.	JHotDraw p.5
Drawing	What: the figure-container interface, aggregating Figures 1..n. Use: the model root holding what is drawn/exported.	JHotDraw p.5
Figure	What: the drawable-element interface; has Handles 1..n and can itself contain figures (the self-association enabling Composite). Use: the extension point for new figure types; the model element a watermark Decorator would wrap.	JHotDraw p.5
Tool	What: the controller interface — the application's current tool. Use: since "features start from controllers," Tools are the usual concept-location entry points; extended via MyTool/YourTool.	JHotDraw p.5
Handle	What: interaction grips on a figure (1..n per Figure) for resize/move. Use: where direct-manipulation behavior lives.	JHotDraw p.5
GoF / [GHJV94]	What: the Gang of Four (Gamma, Helm, Johnson, Vlissides) <i>Design Patterns</i> catalogue — 23 patterns in 3 categories. Use: the shared vocabulary and design templates; "extensibility of frameworks calls for extensive use of design patterns."	JHotDraw p.10, p.16
Creational patterns	What: patterns that initialize/configure classes and objects (control <i>how things get made</i>): Factory Method, Abstract Factory, Builder, Prototype, Singleton. Use: decouple creation from concrete classes (JHotDraw uses Factory Method, Prototype).	JHotDraw p.10
Structural patterns	What: patterns that decouple interface from implementation (control <i>how things are composed</i>): Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy. Use: assemble flexible structures (JHotDraw uses Composite, Decorator).	JHotDraw p.10
Behavioral patterns	What: patterns dealing with dynamic interactions among objects (control <i>how responsibilities/control flow are distributed</i>): includes Strategy, State, Template Method, Command, Observer, etc. Use: make behavior pluggable (JHotDraw uses Strategy, State, Template Method).	JHotDraw p.10

Additional terminology (second tier — verbatim-grounded)

Term	Definition	Source
Dynamic program analysis	Analysis of software performed <i>by executing programs</i> on a real or virtual processor; effective only with sufficient test inputs to produce interesting behavior.	CLLab
Code coverage	A software-testing measure that helps ensure an adequate slice of the program's set of possible behaviors has been observed during dynamic analysis.	CLLab
Controller classes	The classes where features are "often started" — in JHotDraw, the Tool hierarchy (the <<controller>> of MVC); the recommended starting modules for concept location.	CLLab; JHotDraw p.17–18
Initial set of classes	The concept-location output recorded in the lab's two-column table (Domain Class / Responsibility); becomes the initial impact set.	CLLab; SoftwareChange p.9
Domain class	A class that directly realizes a domain concept from the change request — the only kind you keep in the table when a feature touches many classes.	CLLab
GitHub Fork	The team's own copy of the JHotDraw repository, created in the first lab task.	ChangeReq Lab

Term	Definition	Source
GitHub flow	The collaboration workflow each team member follows in future lab exercises (branch/commit/PR discipline on the fork).	ChangeReq Lab
GitHub Projects / TODO Backlog	The board where the user-story card is created and placed — the lab's concrete Product Backlog.	ChangeReq Lab; ChangeInitiation p.7
Portfolio artifact	A lab output explicitly flagged for the mandatory individual portfolio assignment: the user story (ChangeReqLab) and the initial class-set table (CLLab).	ChangeReq Lab; CLLab
Must have / Nice to have / Won't have	The three priority tiers stacked on the Product Backlog "Item" tower.	ChangeInitiation p.7
UMLEditor	The root (client) class of the worked dependency-search CDG — the editor application whose figure-properties change is being located.	ConceptLocation p.12
ClassDiagramGraph	The "most likely supplier" chosen from UMLEditor's five diagram-graph suppliers; marked Propagating after inspection.	ConceptLocation p.14–15
NoteNode	The "wrong way" class — inspected, found not to contain the concept in its composite responsibility, marked Unchanged, triggering backtrack.	ConceptLocation p.16–17
ClassNode	The class where the concept is first located ("Concept location found"); search may extend from it along inheritance.	ConceptLocation p.18–19
RectangularNode / AbstractNode	The generalization chain above ClassNode (<code>ClassNode</code> Δ <code>RectangularNode</code> $\triangleleft$ <code>AbstractNode</code>); AbstractNode is the second location found (highlighted red).	ConceptLocation p.12, p.19–20
MultiLineString / PackageNode / InterfaceNode / ClassRelationshipEdge / SequenceDiagramGraph / UseCaseDiagramGraph / ObjectDiagramGraph / StateDiagramGraph	The remaining named nodes of the UMLEditor CDG — be able to place them on the redrawn graph.	ConceptLocation p.12
Layouter	JHotDraw's name for a Strategy object that arranges a composite figure's children (Java Swing/AWT calls it a layout manager); examples: none, align-to-top, align-to-left.	JHotDraw p.28
SelectTool / SelectToolState	The State-pattern context and abstract state in JHotDraw's selection tool; SelectTool calls <code>state.select()</code> .	JHotDraw p.32
ZeroClickState / OneClickState / TwoClickState	The three concrete SelectToolState subclasses backing the click semantics (1 click = cursor, 2 = word, 3 = whole line).	JHotDraw p.32
LineConnection	The single class holding the invariant figure-connecting algorithm; variant steps are empty methods subclasses overwrite (Template Method).	JHotDraw p.35
DecoratorFigure / BorderDecorator / ShadowDecorator	JHotDraw's Decorator hierarchy over the Figure interface (DecoratorFigure aggregates exactly 1 Figure).	JHotDraw p.39

Term	Definition	Source
GraphicalCompositeFigure	The Composite node of the Figure hierarchy; "delegates methods to all contained Shapes"; enables arbitrary-depth nesting.	JHotDraw p.23
createMenus() / createTools()	The two factory methods of DrawApplication (protected # in the framework, public + in subclasses) that keep menus and tools flexible.	JHotDraw p.43
MyDrawApplication / ClassDiagrammDrawApplication / FigureDrawApplication	Customized application classes that inherit DrawApplication and overwrite its factory methods (class/interface-figure tools vs. circles/boxes tools).	JHotDraw p.43
prototype.clone()	The call by which a tool creates a new figure from its installed prototype (Circle Tool clones a circle, Rectangle Tool a rectangle).	JHotDraw p.46–47
FigureAttributes / fMap	The <<model>> attribute holder behind AbstractFigure: a private fMap:Hashtable with set:void / get:Object.	JHotDraw p.18
AbstractFigure / StandardDrawing / AbstractTool	The default implementations of the Figure, Drawing, and Tool interfaces shown on the MVC class diagram.	JHotDraw p.18
Notify / Update / User Action	The labelled arrows of the MVC interaction triangle: View raises User Action → Controller Updates Model → Model Notifies → View Updates.	JHotDraw p.17
Adapter	Structural, the only dual-scope (class and object) pattern: converts a class's interface into one clients expect so incompatible classes can work together.	JHotDraw p.11, p.13
Bridge	Structural (object): decouple an abstraction from its implementation so the two can vary independently.	JHotDraw p.13
Facade	Structural (object): provide a unified interface to a set of interfaces in a subsystem.	JHotDraw p.13
Flyweight	Structural (object): use sharing to support large numbers of fine-grained objects efficiently.	JHotDraw p.13
Proxy	Structural (object): provide a surrogate or placeholder for another object to control access to it.	JHotDraw p.13
Abstract Factory	Creational (object): an interface for creating families of related/dependent objects without specifying their concrete class.	JHotDraw p.12
Builder	Creational (object): separate construction of a complex object from its representation so one construction process yields different representations.	JHotDraw p.12
Singleton	Creational (object): ensure a class has only one instance and provide a global access point to it.	JHotDraw p.12
Interpreter	Behavioral (class): define a grammar representation for a language plus an interpreter that uses it to interpret sentences.	JHotDraw p.14
Chain of Responsibility	Behavioral (object): decouple sender from receiver by passing the request along a chain of potential handlers until one handles it.	JHotDraw p.14

Term	Definition	Source
Command	Behavioral (object): encapsulate a request as an object — enabling parameterized clients, queued/logged requests, and undo.	JHotDraw p.14
Iterator	Behavioral (object): access an aggregate's elements sequentially without exposing its underlying representation.	JHotDraw p.14
Mediator	Behavioral (object): an object that encapsulates how a set of objects interact.	JHotDraw p.14
Purpose (pattern axis)	The what-the-pattern-does axis: Creational / Structural / Behavioral.	JHotDraw p.11
Scope (pattern axis)	The how-variation-is-fixed axis: Class (inheritance, compile time) vs. Object (composition, run time).	JHotDraw p.11

Tertiary terms (small but quotable)

Term	Definition	Source
Cut-over / go live	The labels on the SC definition slide's graphic: the old-system arrow "cut-over" point and the new system "go live" — the visual of change carrying software from one running version to the next.	SoftwareChange p.2
Walkthrough	The human-review component of Verification, listed alongside functional/unit/structural testing.	SoftwareChange p.13
Baseline	The committed, built state produced by Conclusion ("build the new baseline") that the next change starts from.	SoftwareChange p.14
Wish list	The slide's own quoted nickname for the Product Backlog.	ChangeInitiation p.7
Sticky-note template	The handwritten card graphic "As a [who], I want [what], Because [why]" shown on both the triggers and requirements-form slides.	ChangeInitiation p.3–4
3" × 5" card	The physical size limit a user story must fit; oversized functionality is split into several stories.	ChangeInitiation p.4
Starting modules	The first candidate set the algorithm (Computer lane) finds before any selection — features often start from controller classes.	ConceptLocation p.10, p.21; CLLab
Supplier modules	The set found when the concept is in the composite (not local) responsibility — the descend targets.	ConceptLocation p.10
Backtrack modules	The set found when the concept is absent from the composite responsibility — the back-up targets.	ConceptLocation p.10
Composite responsibility	The phrasing the decision diamond uses for composite functionality ("Is the concept implemented in the composite responsibility?").	ConceptLocation p.10–11
Execution traces	The parenthetical the methodologies slide attaches to dynamic search — the record of what code runs.	ConceptLocation p.7
Figure properties	The concept being located in the entire UMLEditor trace (per the start slide's title "Locating Figure Properties").	ConceptLocation p.12
IDE Debugger	The lab's concrete dynamic-analysis instrument for localizing the classes a change request involves.	CLLab

Term	Definition	Source
Existing features (list)	The linked list in ChangeReqLab from which students pick the JHotDraw feature whose user story they write.	ChangeReqLab
Frame / Panel	The two Java AWT superclasses in the architecture: DrawApplication extends Frame; StandardDrawingView extends Panel.	JHotDraw p.5
DrawingWindow	The second view element named beside DrawingView in JHotDraw's MVC mapping.	JHotDraw p.17
FillColor / Position	The two example figure attributes named on the MVC slide (stored via FigureAttributes' fMap).	JHotDraw p.17–18
FigureEnumeration	The return type of <code>Drawing.figures</code> on the MVC class diagram — how a drawing exposes its figures without exposing its container.	JHotDraw p.18
Selection / notification	The two labelled associations of StandardDrawingView: it holds the selection and receives notification from the Drawing.	JHotDraw p.5
Subclass explosion	Decorator's use-when rationale: "a large number of independent extensions ... would produce an explosion of subclasses."	JHotDraw p.37
getDecoratedType()	The example access method without which a decorated object's type is difficult to obtain (instanceof does not work through the wrapper).	JHotDraw p.39
Network layouts / Gantt diagrams	The two example application domains named for JHotDraw's "technical and structured graphics."	JHotDraw p.4

Compare & Contrast Tables (Exam Drill)

Easily-confused pairs are classic exam material. Every row below is grounded in the slides cited.

Strategy vs. State vs. Template Method

	Strategy	State	Template Method
Category / scope	Behavioral, object	Behavioral, object	Behavioral, class
Intent (slide wording)	Family of algorithms, encapsulated, interchangeable; algorithm varies independently of clients (JHotDraw p.25)	Object alters behavior when internal state changes; "appears to change its class" (JHotDraw p.30)	Skeleton of an algorithm in one operation; subclasses redefine steps without changing structure (JHotDraw p.34)
Variation mechanism	Swap the strategy object held by the Context	Swap the state object; the object itself transitions between states	Override hook methods in a subclass — fixed at compile time
JHotDraw use	Layouters attached to composite figures (none / align-to-top / align-to-left) (JHotDraw p.28)	Tool modes: zoom in/out, select border/text; SelectTool with Zero/One/TwoClickState (JHotDraw p.32)	LineConnection: invariant connect-figures algorithm, variant steps as empty overridable methods (JHotDraw p.35)

	Strategy	State	Template Method
Choose it when	Many related classes differ only in behavior; variants of an algorithm; hide algorithm data; kill conditionals (JHotDraw p.26)	Behavior depends on state and must change at run time; large multipart state conditionals (JHotDraw p.30)	A framework must enforce one fixed process while applications customize details (JHotDraw p.35)
Key cost	Clients must know the strategies; Strategy↔Context communication overhead; more objects (JHotDraw p.29)	(No cons listed on the slide — pros only: localized state behavior, explicit transitions, shareable state objects) (JHotDraw p.33)	Inheritance-bound: the variation point is fixed per subclass, not swappable at run time (JHotDraw p.11, p.34)
One-line discriminator	<i>Who picks the behavior?</i> The client configures it.	<i>Who picks the behavior?</i> The object's own current mode.	<i>Where does variation live?</i> In subclass overrides of a fixed skeleton.

Composite vs. Decorator

	Composite	Decorator
Category	Structural (object)	Structural (object)
Shape	Tree — a whole containing 1..n parts, nested to arbitrary depth (JHotDraw p.21, p.23)	Chain — wrappers each holding exactly one component (aBorderDecorator → aScrollDecorator → aTextView) (JHotDraw p.36)
Goal	Treat individual objects and compositions uniformly (JHotDraw p.19)	Attach additional responsibilities to one object dynamically (JHotDraw p.36)
JHotDraw realization	GraphicalCompositeFigure delegating to all contained Shapes (JHotDraw p.23)	DecoratorFigure with BorderDecorator / ShadowDecorator (JHotDraw p.39)
Operation forwarding	To all children: <code>forall g in children: g.Operation()</code> (JHotDraw p.20)	To the one wrapped component, plus added behavior: <code>Decorator::Operation(); AddedBehavior();</code> (JHotDraw p.38)
Characteristic trap	Type system can't restrict which components a composite holds — run-time checks needed (JHotDraw p.24)	Decorator and component aren't identical — <code>instanceof</code> fails without <code>getDecoratedType()</code> -style access (JHotDraw p.39–40)
Shared trait	Both implement the same Figure interface as the things they contain/wrap — which is why both slot invisibly into the figure hierarchy. (JHotDraw p.23, p.39)	

Factory Method vs. Prototype

	Factory Method	Prototype
Category / scope	Creational, class	Creational, object
Creation mechanism	Subclass overrides <code>FactoryMethod()</code> → <code>return new ConcreteProduct</code> (JHotDraw p.42)	Clone a configured prototypical instance: <code>prototype.clone()</code> (JHotDraw p.45–46)

	Factory Method	Prototype
JHotDraw use	<code>DrawApplication.createMenus()/createTools()</code> overridden by customized applications (JHotDraw p.43)	Each tool initialized with a prototype figure it clones to create new figures (JHotDraw p.47)
Choose it when	A class can't anticipate what to create; subclasses should specify products; localize which helper subclass is the delegate (JHotDraw p.41)	Classes to instantiate are known only at run time (dynamic loading); avoid a parallel factory hierarchy; few state combinations → install prototypes instead (JHotDraw p.45)
Consequences (slide)	Provides hooks for subclasses; connects parallel class hierarchies (JHotDraw p.44)	Hides concrete product classes (fewer names); clients use application-specific classes unmodified; add/remove products at run time; new objects by varying values (JHotDraw p.48)
One-line discriminator	Variation by subclassing the creator .	Variation by swapping the example object — no new creator class needed.

The two change classifications side by side

Axis	Lientz & Swanson (the <i>why</i>)	Impact on functionality (the <i>how</i>)
Values	Perfective (~50%, credit card), Adaptive (~25%, Y2K), Corrective (~21%, bugs), Preventive (~4%, improve code structure) (SoftwareChange p.3)	Incremental (add new), Contraction (remove obsolete), Replacement (replace existing), Refactoring (change structure, behavior unchanged) (SoftwareChange p.4)
Question answered	What motivated the request?	What happens to observable behavior?
Natural pairings	Perfective → usually Incremental; Preventive → Refactoring; Corrective → usually Replacement of faulty behavior; Adaptive → Replacement/Incremental at the environment boundary	Refactoring is the <i>how</i> of Preventive's <i>why</i> — same activity on two axes
Exam trap	The percentages belong to <i>this</i> axis only — never attach percentages to incremental/contraction/replacement/refactoring	"Without changing behavior" belongs to <i>Refactoring</i> only — never to Replacement

Concept-location methodologies — trade-off matrix

Methodology	Requires	Strength	Weakness	L02 tie-in
Human knowledge	An expert who knows the code	Instant, free	Expert may not exist; memory goes stale	First resort when available (ConceptLocation p.7)
Traceability tools	Maintained requirement↔code links	Direct lookup	Only as good as link-maintenance discipline	(ConceptLocation p.7)
Dynamic search (execution traces)	A runnable build + inputs that trigger the feature	Shows code that <i>actually executes</i> for the feature	Misses paths not triggered; needs sufficient test inputs / coverage	The CLLab IDE-debugger method (ConceptLocation p.7, CLLab)

Methodology	Requires	Strength	Weakness	L02 tie-in
Static: dependency search	A CDG extracted from code; ability to judge local vs. composite functionality	Follows structure, finds renamed concepts; systematic via marks	Human judgment per class; can wander without the marks discipline	The UMLEditor algorithm (ConceptLocation p.9–21)
Static: pattern matching (GREG)	Just the source text	Zero setup, instant seeding	Blind to dependencies and renaming (name-edge trap)	Iterative query-refine loop (ConceptLocation p.8)
Static: information retrieval	Indexed/ranked source text	Ranks candidates by textual relevance	Still text-based — same renaming blindness	Listed as the third static technique (ConceptLocation p.7)

The dependency-search decision table

Question (asked in order)	Answer	Action	Mark consequence
Is the concept implemented in the module (its local functionality)?	Yes	Stop the search — location found	(Found; on the trace, the green/red "found" highlight)
Is the concept implemented in the module's composite responsibility ?	Yes	Find the set of supplier modules; schedule them; descend	Current class → Propagating (orange); suppliers → Next (green)
Is the concept implemented in the composite responsibility?	No	Find the set of backtrack modules; back up	Current class → Unchanged (grey)
(Before any inspection)	—	Class neither inspected nor scheduled	Blank (white)

(ConceptLocation p.10–11) In the interactive tool the **Computer** lane owns the three "find set of ..." actions and the **Programmer** lane owns the two judgment diamonds — division of labor between mechanical graph work and semantic judgment. (ConceptLocation p.21)

Prefactoring vs. Postfactoring

	Prefactoring	Postfactoring
Timing	Before the new functionality is written	After the change is actualized
Purpose	Localize (minimize) the impact of the upcoming SC	Eliminate anti-patterns the change introduced
Named examples	Extract Class (gather fields/methods/snippets into a new component class); Extract Superclass (create a new abstract class) (SoftwareChange p.10)	Long method (a method now doing too much); Bloated class (a class grown too large) (SoftwareChange p.12)
Shared definition	Both are refactoring: structure changes, externally observable behavior identical (SoftwareChange p.4)	
Verification proof	Existing tests still pass — before <i>and</i> after each	

Dynamic search vs. static dependency search (the two L02 deep dives)

	Dynamic (IDE debugger / traces)	Static (dependency search)
Program runs?	Yes — on a real or virtual processor (CLLab)	No — works on the CDG extracted from code (ConceptLocation p.9)
Evidence	Which classes actually execute when the feature is exercised	Which classes structurally could implement the concept, judged by reading code/documentation
Blind spot	Untriggered paths (insufficient test inputs) (CLLab)	Judgment errors on local vs. composite; dead code looks live
Bookkeeping	Breakpoints/observations; keep domain classes only (CLLab)	Blank / Propagating / Unchanged / Next marks (ConceptLocation p.11)
Course artifact	The CLLab Domain Class / Responsibility table	The UMLEditor trace reproduction

Common Pitfalls / Gotchas

- **Don't infer deck content from the filename alone.** Verified mapping: `ChangeLec.pdf` = *Introduction to Software Change* (14p); `ChangeReqLec.pdf` = *Change Initiation* (8p) — the "Req" in `ChangeReqLec.pdf` does *not* mean it is the bigger requirements/change deck. (*SoftwareChange* p.1, *ChangeInitiation* p.1)
- **"JHowDraw" typo.** The watermark change-request slide misspells JHotDraw as "JHowDraw" — same framework. (ConceptLocation p.6)
- **Concept location ≠ impact analysis.** Concept location finds the *starting* snippet/class; impact analysis *grows* that into the full impact set. The classes from concept location are only the **initial** impact set. (*SoftwareChange* p.8–9)
- **Local vs. composite functionality is the decision pivot.** If the concept is in the *composite* responsibility but not *local*, you must descend into suppliers — not stop. Confusing the two breaks the dependency-search algorithm. (ConceptLocation p.9–10)
- **Stop condition.** You stop the search only when the concept is implemented in the **local** (module's own) functionality — "*is the concept implemented in the module? [Yes] → stop.*" (ConceptLocation p.10)
- **Backtracking is expected, not failure.** The canonical UMLEditor trace deliberately takes a "wrong way" (toward NoteNode), marks it Unchanged, and backtracks. (ConceptLocation p.16–17)
- **Marks are precise.** *Propagating* and *Unchanged* both mean "inspected"; the difference is whether the **composite** responsibility contains the concept. *Next* ≠ *Blank* (Next is scheduled; Blank is neither inspected nor scheduled). (ConceptLocation p.11)
- **Refactoring's definition is strict.** Changing structure *without changing behavior*. Pre- and post-factoring are both refactoring — before to minimize impact, after to remove anti-patterns. (*SoftwareChange* p.4, p.10, p.12)
- **User story size rule.** It must fit a 3"×5" card; if it can't, split it into several stories — don't cram a big feature into one. (*ChangeInitiation* p.4)
- **Decorator type-identity trap.** With Decorator, Java's `instanceof` won't see the wrapped figure's true type unless exposed (e.g., `getDecoratedType()`); a decorator and its component "aren't identical." (JHotDraw p.39–40)
- **Composite type-safety trap.** Composite can't use the type system to restrict which components a composite holds — enforce with run-time checks. (JHotDraw p.24)

- **MVC roles in JHotDraw.** Model = Figures/Drawing; View = DrawingView/Window; Controller = Tools. Don't put Tools in the model. (JHotDraw p.17)
- **Verification spans phases.** It is not a single final step — it runs vertically alongside Prefactoring → Conclusion. (SoftwareChange p.5, p.13)
- **Reading-key mapping.** The slides cite *Rajlich Ch.6 (2012)* and [GHJV94]; this guide maps those to the course keys [Raj13] and [GHJV94]. Don't expect the literal strings [Raj13] / [MC09] / [Martin] on the slides. (Grounding note)

Additional verified gotchas (file mapping, citation keys, slide quirks)

- **File-to-deck mapping — corrected and verified against the PDFs.** `ChangeLec.pdf` opens with the title slide *Introduction to Software Change (SB5-MAI)* and runs **14** slides; `ChangeReqLec.pdf` opens with *Change Initiation (SB5-MAI)* and runs **8** slides. The grounding note in this guide's header states this same verified mapping, and the deck-name citations (SoftwareChange p.X) / (ChangeInitiation p.X) are correct throughout the body. (SoftwareChange p.1, ChangeInitiation p.1)
 - **[GHJV95] is the literal slide key.** Slide 16 of the JHotDraw deck cites "*Design Patterns [GHJV95]*", not [GHJV94]; both keys denote the same Gamma/Helm/Johnson/Vlissides book. (JHotDraw p.16)
 - **ConceptLocation page count.** The deck's footers read "X / 22" but the file contains 21 physical pages; nothing is missing mid-deck — the Interactive Tool slide (p.21) is the last content slide. (ConceptLocation p.21)
 - **Green does double duty in the trace.** Green is the *Next* mark in the marks table, but the "Concept location found" slide also highlights `ClassNode` in green, and the *second* location (`AbstractNode`) is highlighted **red** instead. Read the slide titles, not just the colors. (ConceptLocation p.11, p.18, p.20)
 - **The trace follows inheritance edges too.** The extension steps run `ClassNode` → `RectangularNode` → `AbstractNode` along generalization arrows — dependency search is not limited to use-dependencies. (ConceptLocation p.19–20)
 - **JHotDraw deck tail.** p.49 is the "Other Design Patterns" section divider and p.50 is blank apart from the page number — there is no additional pattern content to study after Prototype. (JHotDraw p.49–50)
 - **Factory-method visibility flips in the UML.** `DrawApplication` declares `#createMenus` / `#createTools` (protected, #), while the customized subclasses show them as `+` (public) — a small but quotable detail of the slide's class diagram. (JHotDraw p.43)
 - **State's method name changes between GoF and JHotDraw.** The GoF structure names the state operation `Handle()` (with the `state->Handle()` note on `Request()`), but JHotDraw's `SelectTool` realization calls `state.select()` — same delegation, different name. (JHotDraw p.31–32)
 - **ClassDiagrammDrawApplication really has a double m** on the slide — quote it as printed if asked to name the customized application classes. (JHotDraw p.43)
 - **Lab handout typos.** `ChangeReqLab` writes "feature" and "a an artifact"; `CLLab` writes "to the to domain concepts" — the content is unaffected, but the verbatim strings differ from clean English if you search for them. (ChangeReqLab, CLLab)
 - **Team vs. individual in the lab.** The GitHub fork is created per *team*, but the user story is a *mandatory individual* portfolio artifact — don't conflate the two scopes. (ChangeReqLab)
 - **MVC arrow directions.** On the interaction triangle the View never updates the Model directly: User Action goes View→Controller, Update goes Controller→Model (and Controller→View), Notify goes Model→Controller/observers. Drawing a direct View→Model edit arrow is a classic error. (JHotDraw p.17)
-

Exam Focus

- **Order the eight phases** and label each as world-interaction / design / implementation; know that **Verification** spans implementation phases. (SoftwareChange p.5)
- **Lientz & Swanson categories** with their ~percentages and an example each (Perfective/credit card 50%, Adaptive/Y2K 25%, Corrective/bugs 21%, Preventive/structure 4%). (SoftwareChange p.3)
- **Four impacts on functionality** (incremental, contraction, replacement, refactoring) — and the refactoring definition "without changing behavior." (SoftwareChange p.4)
- **Write a correct user story** in the "As a ... I want ... so that ..." form and state the 3"×5"-card / split rules. (ChangeInitiation p.4–6)
- **Define concept location** and explain the domain-concept ↔ code vocabulary gap and the "paste a text" example. (ConceptLocation p.2)
- **Concept triangle:** label Name/Intension/Extension and the edges naming, recognition/location, annotation/traceability; give the Dog/Pes/Hund example. (ConceptLocation p.4–5)
- **Extract concepts** from a given change request (be ready to underline domain nouns like Export/Drawing/Text/Format). (ConceptLocation p.6)
- **Four methodologies** (human knowledge, traceability, dynamic/execution traces, static: dependency/pattern-matching/IR) and what GREP is. (ConceptLocation p.7–8)
- **Run the dependency-search algorithm** on a CDG: starting modules → select → local? → composite? → suppliers vs. backtrack → stop; and apply the **Blank/Propagating/Unchanged/Next** marks. Be ready to reproduce the **UMLEditor trace** with its wrong-way + backtrack. (ConceptLocation p.10–20)
- **Local vs. composite functionality** distinction. (ConceptLocation p.9)
- **Hand-off to Impact Analysis:** concept-location classes = initial impact set. (SoftwareChange p.9)
- **JHotDraw architecture:** name DrawApplication, StandardDrawingView, Drawing, Figure, Tool, Handle and their roles/cardinalities; state it's MVC. (JHotDraw p.5, p.17)
- **GoF essentials:** 23 patterns in 3 categories; the three underlying principles ("program to an interface...", "favor composition...", "find what varies..."); "extensibility of frameworks calls for extensive use of design patterns." (JHotDraw p.9–10, p.16)
- **Eight JHotDraw patterns** with their JHotDraw role: MVC, Composite (figures-in-figures), Strategy (layouters), State (tool modes/click states), Template Method (LineConnection), Decorator (border/shadow), Factory Method (createMenus/createTools), Prototype (tool clones figure). Be able to give intent + JHotDraw example + one pro/con. (JHotDraw p.16–48)
- **Lab deliverables:** a JHotDraw user story (ChangeReqLab) and an initial Domain-Class/Responsibility table from IDE-debugger concept location (CLLab) — both portfolio artifacts.

Self-test drill (questions with slide-grounded answers)

1. **Q:** Define software change in one sentence. **A:** The process of adding new functionality to existing code — the foundation of software evolution and servicing. (SoftwareChange p.2)
2. **Q:** Give the four Lientz & Swanson categories with their percentages and the slide's example for each. **A:** Perfective 50% (credit card), Adaptive 25% (Y2K), Corrective 21% (bugs), Preventive 4% (improve code structure). (SoftwareChange p.3)
3. **Q:** Which impact-on-functionality kind leaves externally observable behavior identical? **A:** Refactoring — changing software structure *without changing behavior*. (SoftwareChange p.4)
4. **Q:** Order the phases and state where Verification sits. **A:** Initiation → Concept Location → Impact Analysis → Prefactoring → Actualization → Postfactoring → Conclusion, with Verification running vertically alongside Prefactoring through Conclusion. (SoftwareChange p.5)

5. **Q:** Which phases are "interactions with the world," which "SC design," which "SC implementation"? **A:** World: Initiation, Conclusion (orange). Design: Concept Location, Impact Analysis (white). Implementation: Prefactoring, Actualization, Postfactoring (green). (SoftwareChange p.5)
6. **Q:** Name the four change-initiation triggers. **A:** User reports a software bug; user asks for an enhancement; programmer proposes improvement; manager wants to meet competitor's functionality. (ChangeInitiation p.3)
7. **Q:** Give both user-story templates from the deck. **A:** "As a [user type], I want [some goal] so that [some reason]." and "As a [user type], I want [some goal] because [why]." (ChangeInitiation p.5)
8. **Q:** What is the physical size rule for a user story and what do you do when it doesn't fit? **A:** It must fit on a 3" × 5" card; functionality that cannot fit must be divided into several user stories. (ChangeInitiation p.4)
9. **Q:** Name the three Product Backlog tiers and the two reasons the backlog keeps changing. **A:** Must have / Nice to have / Won't have; additional knowledge is acquired by the users, and additional clarification is needed by the developers. (ChangeInitiation p.7)
10. **Q:** Define concept location and give the deck's example request. **A:** It finds the code snippet where a change is to be made; example: "Correct error that arises when trying to paste a text." (ConceptLocation p.2)
11. **Q:** Name the concept triangle's corners and all five edge labels. **A:** Corners: Name, Intension, Extension. Edges: naming and definition (Name–Intension), annotation and traceability (Name–Extension), recognition and location (Intension–Extension). (ConceptLocation p.4)
12. **Q:** In the Dog/Pes/Hund example, what is the intension and what is the extension? **A:** Intension: "Hairy animal with teeth..."; extension: the set of instances — Fido, Lassie, Buck (from *Call of the Wild* by Jack London), the pictured dogs. (ConceptLocation p.5)
13. **Q:** Which four concepts are extracted from the watermark change request? **A:** Export, Drawing, Text, Format. (ConceptLocation p.6)
14. **Q:** List the four concept-location methodologies and the three static-search techniques. **A:** Human knowledge; traceability tools; dynamic search (execution traces); static search — dependency search, pattern matching, and information retrieval techniques. (ConceptLocation p.7)
15. **Q:** What does GREP stand for and what does it do? **A:** "Global regular expression print" — prints the lines containing a match for a regular expression; the programmer iteratively formulates queries and investigates results. (ConceptLocation p.8)
16. **Q:** Define local vs. composite functionality. **A:** Local: concepts actually implemented in the module, not delegated to others. Composite: the complete functionality of a module combined with all its supporting modules. Both determined by reading code and documentation. (ConceptLocation p.9)
17. **Q:** State the algorithm's two decision questions and the action for each answer. **A:** "Is the concept implemented in the module?" Yes → stop the search. No → "Is the concept implemented in the composite responsibility?" Yes → find the set of supplier modules; No → find the set of backtrack modules. (ConceptLocation p.10)
18. **Q:** Define all four marks. **A:** Blank — never inspected, not scheduled. Propagating — inspected; composite responsibility contains the concept. Unchanged — inspected; composite responsibility does not contain the concept. Next — scheduled for inspection. (ConceptLocation p.11)
19. **Q:** In the UMLEditor trace, which class is the wrong way, which is the first location, which is the second? **A:** NoteNode (marked Unchanged, backtrack); ClassNode (concept location found); AbstractNode (another location, reached via RectangularNode). (ConceptLocation p.16–20)
20. **Q:** In the interactive concept-location tool, what does the Computer do and what does the Programmer do? **A:** Computer: find the set of starting modules, supplier modules, and backtrack modules. Programmer: select a module and judge whether the concept is implemented in the module / in the composite responsibility. (ConceptLocation p.21)

21. **Q:** What feeds the initial impact set, and what grows it? **A:** The classes identified in concept location make up the initial impact set; class dependencies are analyzed and impacted classes are added to it. (SoftwareChange p.9)
22. **Q:** Name the two prefactoring refactorings and the two postfactoring anti-patterns on the slides. **A:** Prefactoring: Extract Class, Extract Superclass. Postfactoring: long method, bloated class. (SoftwareChange p.10, p.12)
23. **Q:** What two phenomena govern Actualization's neighbor updates? **A:** Change propagation and the ripple effect. (SoftwareChange p.11)
24. **Q:** Name the three testing kinds plus the human technique under Verification. **A:** Functional, unit, and structural testing; walkthroughs. (SoftwareChange p.13)
25. **Q:** What are Conclusion's three actions? **A:** Commit finished code into version control; build the new baseline; prepare for the next change. (SoftwareChange p.14)
26. **Q:** What is JHotDraw for, and what is its origin story? **A:** A framework for drawing technical and structured graphics (network layouts, Gantt diagrams); originally developed in Smalltalk by Kent Beck and Ward Cunningham; HotDraw was one of the first projects explicitly designed for reuse and labelled a framework. (JHotDraw p.4)
27. **Q:** Recite the six core classes with their AWT parents and cardinalities. **A:** DrawApplication (extends Frame) 1—1 StandardDrawingView (extends Panel); DrawApplication owns the current Tool (1) and a Drawing; Drawing (figure container) holds Figure 1..n; Figure has Handle 1..n; StandardDrawingView holds the selection and receives notification. (JHotDraw p.5)
28. **Q:** Give the three GoF principles. **A:** Program to an interface, not to an implementation; favor composition over class inheritance; find what varies and encapsulate it. (JHotDraw p.9)
29. **Q:** Which pattern appears in both Class and Object scope? **A:** Adapter. (JHotDraw p.11)
30. **Q:** List the eight patterns JHotDraw makes extensive use of, and the deck's general observation. **A:** Model-View-Controller, Composite, Strategy, State, Template Method, Decorator, Factory Method, Prototype; "Extensibility of frameworks calls for extensive use of design patterns." (JHotDraw p.16)
31. **Q:** Map JHotDraw onto MVC. **A:** Model: Figures (attributes like FillColor, Position) and Drawing (figure container). View: DrawingView (clipping view of a window), DrawingWindow. Controller: Tools that manipulate the model. (JHotDraw p.17)
32. **Q:** What are the three click states of SelectTool and their text-field semantics? **A:** ZeroClickState, OneClickState, TwoClickState — 1 click sets the cursor, 2 clicks select a word, 3 clicks select the whole line. (JHotDraw p.32)
33. **Q:** Which class holds the Template Method for connections, and what are the two example connection semantics? **A:** LineConnection; Petri net connections vs. class diagram connections. (JHotDraw p.35)
34. **Q:** State the Decorator type-identity problem precisely. **A:** The type of the decorated object is difficult to obtain if it is not exposed by access methods like getDecoratedType() (Java's instanceof does not work) and no separate list of all figures including the decorated ones exists. (JHotDraw p.39)
35. **Q:** Name JHotDraw's two factory methods and the two customized application classes on the slide. **A:** createMenus() and createTools(); ClassDiagrammDrawApplication (tools for class and interface figures) and FigureDrawApplication (tools for circles, boxes, etc.). (JHotDraw p.43)
36. **Q:** Give all four Prototype consequences from the slide. **A:** Hides concrete product classes (fewer names for clients); lets clients work with application-specific classes without modification; lets you add and remove products at run time; lets you specify new objects by varying values. (JHotDraw p.48)
37. **Q:** What are the two CLLab objectives and the exact portfolio table format? **A:** Apply the IDE Debugger to locate feature concepts at runtime; create the list of the initial set of classes — recorded as a two-column table: Domain Class | Responsibility. (CLLab)

38. **Q:** What five classwork tasks does ChangeReqLab list? **A:** Team forks the JHotDraw repository on GitHub; members follow GitHub flow in future labs; select an existing JHotDraw feature and write its user story (from the linked existing-features list); use GitHub Projects; create a card for the user story in the TODO Backlog. (ChangeReqLab)
39. **Q:** State the four Strategy use-when conditions. **A:** Many related classes differ only in their behavior; you need different variants of an algorithm (e.g., space/time trade-offs); an algorithm uses data clients shouldn't know about; a class defines many behaviors appearing as multiple conditional statements in its operations. (JHotDraw p.26)
40. **Q:** State the three Decorator use-when conditions. **A:** To add responsibilities to individual objects dynamically and transparently (without affecting other objects); for responsibilities that can be withdrawn; when extension by subclassing is impractical because many independent extensions would produce an explosion of subclasses. (JHotDraw p.37)
41. **Q:** Give Composite's three pros and its con. **A:** Pros: wherever client code expects a primitive object it can also take a composite; clients are simpler (uniform treatment); easier to add new kinds of components without changing clients. Con: the design can become overly general — you can't rely on the type system to restrict a composite's components and must use run-time checks. (JHotDraw p.24)
42. **Q:** What is the partial-comprehension analogy, and what three coping strategies does the slide list? **A:** Visiting a large city; minimum essential understanding, an as-needed strategy, and understanding how certain specific concepts are reflected in the code. (ConceptLocation p.3)

Scope limits — what these slides deliberately do NOT cover

Knowing the boundaries of the source material protects you from over-claiming "the slides say..." in an exam answer:

- **No cons for State.** The State Consequences slide lists pros only (localized state behavior, explicit transitions, shareable state objects); unlike Composite, Strategy, and Decorator there is no Cons block. (JHotDraw p.33)
 - **No consequences slide labelled pros/cons for Factory Method or Prototype** — Factory Method gets two unlabelled consequences (hooks for subclasses; connects parallel hierarchies, JHotDraw p.44) and Prototype gets four (JHotDraw p.48), none framed as drawbacks.
 - **No GoF structure diagram for Template Method** — it is the only deep-dived pattern given intent (p.34) and JHotDraw example (p.35) but no UML structure slide. (JHotDraw p.34–35)
 - **No intents for Memento, Observer, Visitor** anywhere in the deck — they exist only as names in the Purpose × Scope table. (JHotDraw p.11)
 - **"Other Design Patterns" is an empty section** — a divider slide with nothing after it; do not expect Adapter/Observer/Command examples from this lecture. (JHotDraw p.49–50)
 - **The Impact Analysis, Prefactoring, Actualization, Postfactoring, Verification, and Conclusion phases get exactly one preview slide each** in Lo2 — their full treatment belongs to later lectures; Lo2 only owns Initiation and Concept Location in depth. (SoftwareChange p.9–14)
 - **The UML Editor trace never names the located change** beyond "figure properties" — the slides show *where* the search goes, not what edit is eventually made. (ConceptLocation p.12–20)
 - **The labs name only these tools:** JHotDraw, GitHub (Fork, GitHub flow, GitHub Projects, TODO Backlog), and the IDE Debugger — no other tooling appears in the Lo2 handouts. (ChangeReqLab, CLLab)
-

Deck Walkthroughs (Slide-Level Detail)

A compressed but complete narrative of every slide, deck by deck — useful for locating any topic by its slide and for verifying that nothing on a slide is missed.

Introduction to Software Change (ChangeLec . pdf) — 14 slides

- **p.1 Title.** *Introduction to Software Change (SB5-MAI)*, Jan Corfixen Sørensen, University of Southern Denmark.
- **p.2 Software Change.** Definition ("process of adding new functionality to existing code"); "foundation of software evolution, servicing"; illustrated by the old-system arrow cutting over ("cut-over") to a new-system arrow going live ("go live").
- **p.3 Characteristics of SC.** Lientz and Swanson pie chart: Perfective 50%, Adaptive 25%, Corrective 21%, Preventive 4%, with the examples credit card / Y2K / bugs / improve code structure.
- **p.4 Impact on Functionality.** Incremental (adding new functionality), Contraction (removing obsolete functionality), Replacement (replacing existing functionality), Refactoring (changing software structure without changing behavior).
- **p.5 Phased model of SC.** "Main topic of this course"; "Preview of Phases of SC"; the chevron stack with the orange/white/green legend (interactions with the world / SC design / SC implementation) and the vertical VERIFICATION box.
- **p.6 Initiation.** SC starts by a change request; requirements may be a software bug, an enhancement, an improvement; prioritization of change requests, etc.
- **p.7 Design.** The Concept / Impact Set / Software figure with the programmer's Concept Location arrow inward and Impact Analysis arrows outward; Concept Location and Impact Analysis jointly highlighted.
- **p.8 Concept Location.** Concepts are extracted from the change request; extracted concepts are located in the code and used as a starting point of SC.
- **p.9 Impact Analysis.** Determine strategy and impact of change; classes identified in concept location make up the initial impact set; class dependencies are analyzed and impacted classes are added to the impact set.
- **p.10 Prefactoring.** Opportunistic refactoring that localizes (minimizes) impact of SC; Extract Class (Fowler) — gather fields, methods, and code snippets into a new component class; Extract Superclass — create new abstract class.
- **p.11 Actualization.** Creates new code; plugs it into the old code; visit neighboring classes and update them — change propagation, ripple effect.
- **p.12 Postfactoring.** Eliminate any anti-patterns that may have been introduced: long method (after added functionality, some methods may be doing too much), bloated class (a class may be too large).
- **p.13 Verification.** Guarantees correctness of the change; testing — functional, unit, structural; walkthroughs; the VERIFICATION box itself is highlighted.
- **p.14 Conclusion.** Commit finished code into version control; build the new baseline; prepare for the next change.

Change Initiation (ChangeReqLec . pdf) — 8 slides

- **p.1 Title.** *Change Initiation (SB5-MAI)*.
- **p.2 Initiation.** Identical content to SoftwareChange p.6 (change request, bug/enhancement/improvement, prioritization), anchored by the Initiation-highlighted sidebar — the splice point between the two decks.

- **p.3 Change Initiation.** The four triggers (user bug report, user enhancement, programmer improvement, manager meeting competitor functionality) beside the sticky-note graphic "As a [who], I want [what], Because [why]".
- **p.4 Requirements form.** Sentence or paragraph; bug report; user story — with the three sub-rules: limit complexity and potential for misunderstanding, fits on a 3"×5" card, divide into several stories if it cannot fit; same sticky note repeated.
- **p.5 What is a User Story?** The definition ("short and simple descriptions of capabilities written from the perspective of the person who desires the new capability") and the two bold templates ("...so that..." / "...because...").
- **p.6 User Story Example.** The valid-user story (access the system → review my information) and the administrator story (restrict access to valid users → protect user information).
- **p.7 Product Backlog.** "Wish list"; add/delete/modify requirements; additional knowledge acquired by users; additional clarification needed by developers; the Item tower sliced into Must have / Nice to have / Won't have.
- **p.8 Change Initiation (diagram).** Stakeholders → Change Initiation → Product Backlog [Change Request, dashed] → Change → New Code atop Existing Code inside dashed Desired Code.

Concept Location (ConceptLocation.pdf) — 21 slides

- **p.1 Title.** *Concept Location (SB5-MAI).*
- **p.2 Concept Location.** Finds the code snippet where a change is to be made; change requests are most often formulated in terms of domain concepts; example: "Correct error that arises when trying to paste a text"; Concept-Location-highlighted sidebar.
- **p.3 Partial Code comprehension.** Large programs cannot be completely comprehended: minimum essential understanding, as-needed strategy, understand how certain specific concepts are reflected in the code; analogy: visiting a large city (illustrated by a lone figure on a crossing amid towering signage).
- **p.4 Concept Triangle.** Name / Intension / Extension boxes; edge labels naming + definition (Name–Intension), annotation + traceability (Name–Extension), recognition + location (Intension–Extension).
- **p.5 Concept Triangle Example.** Name "Dog / Pes / Hund"; intension "Hairy animal with teeth..."; extensions Fido, Lassie, Buck (in "Call of the wild" by Jack London), plus pictured dogs.
- **p.6 Concept Location Example.** The watermark change request (with export / text / drawings / formats underlined; framework misspelled "JHowDraw"); Concepts: Export, Drawing, Text, Format.
- **p.7 Concept Location Methodologies.** Human knowledge; traceability tools; dynamic search (execution traces); static search: dependency search, pattern matching and information retrieval techniques.
- **p.8 GREP Search Technique.** Acronym; prints lines matching a regular expression; programmer iteratively formulates the query and investigates the results.
- **p.9 Dependency Search Technique.** Uses Class Dependency Graphs (CDG) extracted from the existing code (slide typo: "extracted form the existing code"); local functionality (implemented in the module, not delegated); composite functionality (module plus all its supporting modules); determined by reading code and documentation.
- **p.10 Dependency search (activity diagram).** Find set of starting modules → Select one module → [concept in module? Yes → stop] → [concept in composite responsibility? Yes → find supplier modules / No → find backtrack modules] → loop back to Select one module.
- **p.11 Status of components (marks).** The four-row table: Blank / Propagating / Unchanged / Next, colored white / orange / grey / green.

- **p.12–20 The UML Editor trace.** Nine slides (Start; Classes to inspect; Most likely supplier; Next classes to inspect; Wrong way; Backtrack; Concept location found; Possible extension of the search; Another location found), each a recolored copy of the same CDG, attributed "© 2012 Václav Rajlich, Software Engineering: The Current Practice, Ch. 6" (original slides 31–39). Full color narrative in the Key Concepts section above.
- **p.21 Interactive Tool for Concept Location.** The same activity diagram split into Computer and Programmer swim lanes — Computer finds module sets, Programmer makes the two semantic judgments.

Introduction to JHotDraw Framework (JHotDraw . pdf) — 50 slides

- **p.1 Title; p.2 Outline.** Four parts: JHotDraw; GoF Design Patterns; JHotDraw Design Patterns; Other Design Patterns.
- **p.3 Section divider:** JHotDraw.
- **p.4 The JHotDraw Framework.** Target: drawing technical and structured graphics (network layouts, Gantt diagrams); originally developed in Smalltalk by Kent Beck and Ward Cunningham; HotDraw one of the first projects explicitly designed for reuse and labelled a framework; demo screenshot with Connections, Groups, Text, Annotation, Connected Text, Images, URL Attachments and the Selection Tool status bar.
- **p.5 Understanding JHotDraw (1).** The architecture class model: Java AWT classes Frame and Panel above DrawApplication and StandardDrawingView; DrawApplication 1—1 StandardDrawingView (selection, notification); Drawing 1—1..n Figure (figure container); Figure 1..n Handle; DrawApplication's current tool 1 Tool.
- **p.6 Understanding JHotDraw (2).** The same demo screenshot annotated with the three vocabulary arrows: Tools (the toolbar), Figures (the drawn shapes), Drawing (the canvas).
- **p.7 Using the Framework.** The architecture diagram extended with application-specific classes: MyTool/YourTool under Tool, MyFigure/YourFigure under Figure — the framework's extension points.
- **p.8 Section divider:** GoF Design Patterns.
- **p.9 Principles.** Emphasis on flexibility and reuse through decoupling of classes; program to an interface, not an implementation; favor composition over class inheritance; find what varies and encapsulate it.
- **p.10 General Categories.** 23 patterns in three categories — creational (initializing and configuring classes and objects), structural (decoupling interface and implementation), behavioral (dynamic interactions among societies of classes and objects).
- **p.11 Purpose and Scope.** The full classification table (reproduced in the GoF Catalogue section above).
- **p.12 Creational Patterns.** Verbatim intents: Factory Method (class); Abstract Factory, Builder, Prototype, Singleton (object).
- **p.13 Structural Patterns.** Verbatim intents: Adapter (class/object); Bridge, Composite, Decorator, Facade, Flyweight, Proxy (object).
- **p.14 Behavioral Patterns.** Verbatim intents: Interpreter, Template Method (class); Chain of Responsibility, Command, Iterator, Mediator (object).
- **p.15 Section divider:** JHotDraw Design Patterns.
- **p.16 Design Patterns in JHotDraw.** The eight-pattern list "[GHJV95]" and the general observation that framework extensibility calls for extensive pattern use.
- **p.17 MVC (1).** Model = Figures (FillColor, Position) + Drawing; View = DrawingView (clipping view of a window) + DrawingWindow; Controller = Tools; the Controller/Model/View triangle with Update, Notify, User Action arrows.
- **p.18 MVC (2).** The class diagram with full method signatures (detailed in Pattern Mechanics above).

- **p.19–24 Composite.** Intent and use-when (p.19); GoF structure with forall-children note (p.20); object structure of nested composites (p.21); JHotDraw composite figure of 5 subordinate figures with composition → scaling/rotation (p.22); Figure hierarchy with GraphicalCompositeFigure delegating to contained Shapes (p.23); consequences pros/cons (p.24).
- **p.25–29 Strategy.** Intent (p.25); four use-when bullets (p.26); GoF structure (p.27); JHotDraw layouters with the three layout renderings (p.28); consequences pros/cons (p.29).
- **p.30–33 State.** Intent and use-when (p.30); GoF structure (p.31); JHotDraw tool modes — zoom in/out, select border/text, SelectTool with the three click states (p.32); consequences (pros only) (p.33).
- **p.34–35 Template Method.** Intent (p.34); JHotDraw LineConnection with Petri-net vs. class-diagram connection examples (p.35).
- **p.36–40 Decorator.** Intent with the BorderDecorator/ScrollDecorator/TextView object chain (p.36); three use-when bullets including subclass explosion (p.37); GoF structure with addedState/AddedBehavior notes (p.38); JHotDraw borders and shadows plus the type-identity note (p.39); consequences pros/cons (p.40).
- **p.41–44 Factory Method.** Intent and three use-when bullets (p.41); GoF structure with product=FactoryMethod()/return-new notes (p.42); JHotDraw createMenus()/createTools() hierarchy (p.43); two consequences (p.44).
- **p.45–48 Prototype.** Intent and three use-when bullets (p.45); the Circle/Rectangle Tool clone structure (p.46); the tool-clones-prototype rule (p.47); four consequences (p.48).
- **p.49–50.** "Other Design Patterns" divider; blank closing page.

Source Map

Deck (file)	Pag es	Sections covered
<i>Introduction to Software Change (ChangeLec . pdf)</i>	1	Title
	2	SC definition; evolution/servicing
	3	Lientz & Swanson categories (perfective/adaptive/corrective/preventive + %)
	4	Impact on functionality (incremental/contraction/replacement/refactoring)
	5	Phased model of SC (preview; world/design/implementation grouping)
	6	Initiation
	7	Design (Concept Location + Impact Analysis; impact set figure)
	8	Concept Location
	9	Impact Analysis (initial impact set)
	10	Prefactoring (Extract Class / Extract Superclass)
	11	Actualization (change propagation, ripple effect)
	12	Postfactoring (long method, bloated class)
	13	Verification (functional/unit/structural testing, walkthroughs)
	14	Conclusion (commit, new baseline, next change)
<i>Change Initiation (ChangeReqLec . pdf)</i>	1	Title

Deck (file)	Pag es	Sections covered
	2	Initiation overview (change request, requirements, prioritization)
	3	Change initiation triggers (user/programmer/manager; "As a [who]...")
	4	Requirements form (sentence/paragraph, bug report, user story; 3"×5" card)
	5	What is a user story? (templates)
	6	User story examples (valid user, administrator)
	7	Product Backlog (Must/Nice-to-have/Won't have)
	8	Change initiation end-to-end (Stakeholders → Backlog/Change Request → Change → New/Existing/Desired Code)
<i>Concept Location</i> (<i>ConceptLocation.pdf</i>)	1	Title
	2	Concept location definition; "paste a text" example
	3	Partial code comprehension (as-needed; large-city analogy)
	4	Concept triangle (Name/Intension/Extension; edges)
	5	Concept triangle example (Dog/Pes/Hund)
	6	Concept-location example — JHotDraw watermark change request; concepts Export/Drawing/Text/Format
	7	Methodologies (human/traceability/dynamic/static)
	8	GREP search technique
	9	Dependency search; local vs. composite functionality
	10	Dependency-search activity diagram
	11	Component marks (Blank/Propagating/Unchanged/Next)
	12– 20	UMLEditor dependency-search trace (Start → Inspect → Most likely supplier → Next → Wrong way → Backtrack → Found → Extension → Another location)
	21	Interactive tool (Computer/Programmer swim lanes)
<i>Introduction to JHotDraw</i> <i>Framework</i> (<i>JHotDraw.pdf</i>)	1–3	Title; outline; section divider
	4	JHotDraw framework (purpose; Smalltalk HotDraw / Beck & Cunningham)
	5	Architecture class model (DrawApplication, StandardDrawingView, Drawing, Figure, Tool, Handle)
	6	Demo GUI (Tools/Figures/Drawing)
	7	Using the framework (MyTool/YourTool, MyFigure/YourFigure extension points)
	8– 14	GoF patterns (principles; 3 categories; purpose/scope table; creational/structural/behavioral)
	15– 16	JHotDraw patterns list; "extensibility calls for patterns"
	17– 18	Model-View-Controller in JHotDraw
	19– 24	Composite (intent/structure/JHotDraw figures/consequences)

Deck (file)	Pag es	Sections covered
	25– 29	Strategy (intent/structure/JHotDraw layouters/consequences)
	30– 33	State (intent/structure/JHotDraw tool modes & click states/consequences)
	34– 35	Template Method (intent/JHotDraw LineConnection)
	36– 40	Decorator (intent/use/structure/JHotDraw border&shadow/consequences)
	41– 44	Factory Method (intent/structure/JHotDraw createMenus&createTools/consequences)
	45– 48	Prototype (intent/structure/JHotDraw tool clone/consequences)
	49– 50	"Other Design Patterns" divider (no further content)
<i>CASE Study Lab</i> (<i>ChangeReqLab</i> .pdf)	1	Fork JHotDraw, GitHub flow, write a user story, GitHub Projects card → portfolio
<i>Concept Location Lab</i> (<i>CLLab1</i> .pdf)	1	Dynamic analysis via IDE debugger; produce initial Domain Class / Responsibility table → portfolio

Verified file-to-deck mapping

The authoritative mapping, confirmed by reading each file's title slide and page footers:

File	Title slide reads	Physical pages	Footer count	Citation key used in this guide
<i>ChangeLec</i> .pdf	<i>Introduction to Software Change (SB5-MAI)</i>	14	"X / 14"	(SoftwareChange p.X)
<i>ChangeReqLec</i> .pdf	<i>Change Initiation (SB5-MAI)</i>	8	"X / 8"	(ChangeInitiation p.X)
<i>ConceptLocat ion</i> .pdf	<i>Concept Location (SB5-MAI)</i>	21	"X / 22" (one fewer physical page than the footer count)	(ConceptLocation p.X)
<i>JHotDraw</i> .pdf	<i>Introduction to JHotDraw Framework (SB5-MAI)</i>	50	"X / 50" (p.50 blank)	(JHotDraw p.X)
<i>ChangeReqLab</i> .pdf	<i>[ChangeReqLab] CASE Study Lab</i>	1	—	(ChangeReqLab)
<i>CLLab1</i> .pdf	<i>[CLLab] Concept Location Lab</i>	1	—	(CLLab)

This table agrees with the file attribution in the header grounding note (*ChangeLec* .pdf = *Introduction to Software Change*, *ChangeReqLec* .pdf = *Change Initiation*); all in-body citations by deck name are correct either way. The page numbers cited throughout this guide are the *physical* page positions in each PDF, which coincide with the printed slide numbers in every deck.